音效管理架构
开发指南

版本号: 1.2
发布日期: 2023.09.07

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.06.02	AWA1692 AWA1746 AWA1588	初始版本
1.1	2023.08.10	AWA1692	新增使用流程 新增源文件说明 新增函数接口说明 新增软件框图和流程图 新增库发布版本算法支持列表
1.2	2023.09.07	AWA1692	更新函数接口、源文件列表 新增外部接口使用示例 新增依赖环境说明 新增插件开发说明 新增 buildroot 系统适配

广州星翼电子科技有限公司zdyzlinux

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 相关术语 2

2 音效管理介绍 **3**
2.1 功能说明 3
2.2 音效支持表 3
2.3 组件关系 5
2.4 目录结构 6
2.4.1 音效管理目录 6
2.4.2 远程音效目录 8

3 模块接口说明 **9**
3.1 源文件列表 9
3.1.1 linux 端源码 9
3.1.2 rtos 端源码 11
3.2 软件框图 12
3.3 软件主要流程 13
3.3.1 数据处理流程 13
3.3.2 远程音效流程 14
3.3.3 实时调参流程 15
3.4 关键数据结构 16
3.4.1 sfx_platform 16
3.4.2 sfx_ap_dev 16
3.4.3 sfx_audio_device 16
3.4.4 sfx_plugin 17
3.5 外部接口 18
3.5.1 音效管理接口 18
3.5.1.1 sfxmgmt_init 18
3.5.1.2 sfxmgmt_exit 18
3.5.2 数据流接口 18
3.5.2.1 sfxmgmt_ap_open 18
3.5.2.2 sfxmgmt_ap_close 19
3.5.2.3 sfxmgmt_ap_do 19
3.5.3 实时调参接口 20
3.5.3.1 sfxmgmt_parm_adjust_server 20
3.5.3.2 sfxmgmt_parm_setup 20

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.5.3.3 sfxmgmt_parm_setup_done 20
3.5.3.4 sfxmgmt_parm_setup_sync 21
3.5.3.5 sfxmgmt_parm_setup_byreinit 21
3.5.3.6 sfxmgmt_parm_setup_byreinit_done 21
3.5.3.7 sfxmgmt_parm_setup_byreinit_sync 22
3.6 内部接口 22
3.6.1 音效插件接口 (主机-linux) 22
3.6.1.1 sfx_plugin adapter 22
3.6.1.2 sfx_plugin register 23
3.6.2 音效插件接口 (远程-rtos) 24
3.6.2.1 sfx_plugin adapter 24
3.6.3 硬件音效 IO 接口 24
3.6.3.1 hw_sfx_read 24
3.6.3.2 hw_sfx_write 25
3.6.3.3 hw_sfx_update_bits 25
3.6.4 远程音效 IO 接口 25
3.6.4.1 rpsfx io adapter 25
3.7 调试接口 26
3.7.1 文件保存 26
3.7.2 耗时调试 26

4 应用使用方法 27
4.1 依赖环境配置 27
4.2 直接使用 28
4.3 集成 tinyalsa 使用 28
4.4 集成 alsalib 使用 28
4.5 外部接口使用示例 29
4.6 配置文件使用 29
4.6.1 音效设备描述文件 29
4.6.1.1 文件格式说明 30
4.6.1.2 文件示例 31
4.6.1.3 文件修改示例 32
4.6.2 音效参数配置文件 33
4.6.2.1 文件格式说明 33
4.6.2.2 文件示例 34
4.7 测试 demo 36
4.8 调参 demo 36

5 插件开发 (基于音效管理库) 37
5.1 软件类音效插件开发 37
5.2 硬件类音效插件开发 40
5.3 远程类音效插件开发 40

6 上位机实时调参 41

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.1 前置条件 41
6.2 音效算法调参 41
6.2.1 图形化调参 42
6.2.2 下发参数 44
6.3 音效调参效果确认 44

7 音效算法调参说明 46
7.1 软件类 46
7.1.1 EQ 46
7.2 硬件类 50
7.2.1 DRC 50
7.2.2 HPF 53

8 FAQ 56
8.1 Android 系统中上位机设置参数失败 56

广州星翼电子科技有限公司zdyzlinux

插　图

图 2-1　sfx_mgmt 组件关系 5
图 3-1　sfx_mgmt 框图 12
图 3-2　sfx_mgmt 数据处理流程 13
图 3-3　sfx_mgmt 远程音效 14
图 3-4　sfx_mgmt 实时调参 15
图 6-1　tigerAACT 界面-连接 42
图 6-2　HW-HPF 界面示意图 43
图 7-1　EQ 调参界面 47
图 7-2　EQ 处理前信号 48
图 7-3　EQ 的参数配置 49
图 7-4　EQ 处理后效果 50
图 7-5　HW-DRC 调参界面 51
图 7-6　DRC 参数曲线 52
图 7-7　DRC 输入信号 52
图 7-8　DRC 输出信号 53
图 7-9　HW-HPF 调参界面 53
图 7-10　HPF 输入信号 54
图 7-11　HPF 输出信号 55

广州星翼电子科技有限公司zdyzlinux

表　格

表 1-1　适用产品列表 2
表 1-2　硬件术语 2
表 1-3　软件术语 2
表 2-1　音效算法支持列表 v1.0.4 3
表 2-2　音效算法支持列表 v2.6.0 4
表 2-3　音效算法支持列表 v2.9.2 4
表 4-1　音效设备描述文件格式参数 30
表 6-1　图形化调参支持列表 43

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档为 sunxi 平台音效管理架构开发指南文档，该音效管理架构结合 AACT 上位机可为使用者提供针对音效算法的快捷使用和管理。其中该音效管理架构可为不同使用场景配置不同音效，从而按需优化各场景声音，同时该音效架构可采用硬件音效算法、多核异构处理算法，进一步节省 cpu 资源消耗。主要作用示例如下。

主要作用 1：通过配置文件对不同音频应用场景嵌入不同的音效组合，且支持不同音效参数。

如系统声音通过扬声器播放场景，使用 EQ + DRC 的音效组合，其中 EQ 参数设置为补偿功放芯片的频响曲线为直线，DRC 参数设置为压缩高峰值音量为扬声器可接受振幅范围内。整体达到还原真实声音和延长扬声器寿命的效果。

再如语音通话通过麦克风录音场景，使用 AGC + EQ + ANS 的音效组合，其中 AGC 参数设置为加大小音量幅值，压缩大音量幅值，使录音声音大小保持平缓，EQ 参数设置为增强人声频段幅值，在背景音中提高人声，ANS 参数设置为降低环境音噪声，整体达到语音通话时保证我方录音的人声清晰，无嘈杂背景音的效果。

在此基础上，若 soc 支持硬件 DRC 算法，或有 DSP 等异构核，则可将上述中的 DRC 算法替换为硬件 DRC，EQ 算法放置于 DSP 核处理，减少音效算法大量计算对 cpu 资源的消耗。

主要作用 2：结合 AACT 上位机针对指定音频应用场景的音效组合、指定音效算法进行实时调参。

该作用用于提高调参效率。当确认好某个音频应用场景的音效组合后，可使用上位机工具选择具体的音效算法，并进行图形化调参，确认参数后可实时将参数下发至板卡实时生效，从而快速确认最优参数。

1.2 目标读者

音频系统相关人员。

1.3 适用范围

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 1-1: 适用产品列表

产品名称	系统版本
A523	Android13
T527	Android13
T113	buildroot

1.4 相关术语

表 1-2: 硬件术语

相关术语	解释说明
HW-DRC	硬件 DRC 算法，将 DRC 集成到音频 IP 的算法
HW-HPF	硬件 HPF 算法，将 HPF 集成到音频 IP 的算法
HW-EQ	硬件 EQ 算法，将 EQ 集成到音频 IP 的算法

表 1-3: 软件术语

相关术语	解释说明
AACT	音频调试上位机工具
sfx_mgmt	音效管理架构
AGC	自动增益控制算法，一般用于统一音频幅值
ANS	降噪算法，一般用于前处理降噪
AEC	回声消除算法，一般用于通话场景
DRC	动态换为压缩算法，一般用于动态调整音频输出幅值
HPF	高通滤波算法，一般用于过滤极低频的次声波
EQ	均衡器算法，一般用于音效调节

音效设备

在音效管理架构中，音效设备特指某个**音频使用场景**和某个**音频硬件设备**的结合或单独一个，其中音频使用场景如系统声音播放、语音通话场景等，音频硬件设备如扬声器、耳机、麦克风等。

如 “Speaker” 表示音频硬件设备扬声器。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 音效管理介绍

2.1 功能说明

音效管理架构，可对软件类、硬件类、远程处理 (多核异构) 类音效进行实时管理，具体如下。

- 支持根据不同音效设备及应用场景使用不同音效组合及音效参数；
- 支持软件类音效、硬件类音效、远程处理类音效使用；
- 支持以描述文件形式配置所需音效算法组合；
- 支持以描述文件形式配置音效算法参数；
- 支持某个音效组合的某个音效参数实时调参；
- 支持上位机图形化实时调参、离线实时调参；
- 支持指定一个或多个非硬件类音效处理后数据保存；
- 支持音效算法耗时测试、架构耗时测试。

2.2 音效支持表

版本号：**sfxmgmt-v1.0.4**

表 2-1: 音效算法支持列表 v1.0.4

算法类型	算法名称	算法版本	功能	上位机调参
硬件类	DRC	1.0.0	音量动态范围控制	√
	HPF	1.0.0	高通滤波器	√
软件类	EQ	1.0.0	均衡器	√
	AGC	1.0.0	自动增益控制	
	ANS	NULL	降噪	

- 支持根据不同音效设备及应用场景使用不同音效组合及音效参数;
- 支持软件类音效、硬件类音效类音效使用;
- 支持以描述文件形式配置所需音效算法组合;
- 支持以描述文件形式配置音效算法参数;
- 支持某个音效组合的某个音效参数实时调参;
- 支持上位机图形化实时调参、离线实时调参;
- 支持测试 demo。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

版本号：**sfxmgmt-v2.6.0 & sfxmgmt_rp-v1.1.2**

表 2-2: 音效算法支持列表 v2.6.0

算法类型	算法名称	算法版本	功能	上位机调参
硬件类	DRC	1.0.0	音量动态范围控制	√
	HPF	1.0.0	高通滤波器	√
软件类	EQ	1.0.0	均衡器	√
	AGC	NULL	自动增益控制	
	ANS	NULL	降噪	
	AEC	NULL	回声消除	
	ROUTE	1.0.0	通道映射	
	VOLUME	1.0.0	通道音量调节	
	PHASE	1.0.0	通道相位时延调节	
	PASS FILTER	1.0.0	高/低通滤波器	
远程类	EQ	1.0.0	均衡器	√
	ANS	NULL	降噪	
	AEC	NULL	回声消除	

- 支持远程处理 (多核异构) 类音效；
- 新增软件算法；
- 支持软件类算法、远程类算法处理后文件保存；
- 支持版本查看；
- 支持 menuconfig 配置化编译。

版本号：**sfxmgmt-v2.9.2 & sfxmgmt_rp-v2.0.4**

表 2-3: 音效算法支持列表 v2.9.2

算法类型	算法名称	算法版本	功能	上位机调参
硬件类	DRC	1.0.0	音量动态范围控制	√
	HPF	1.0.0	高通滤波器	√
软件类	EQ	1.1.0	均衡器	√
	AGC	NULL	自动增益控制	
	ANS	NULL	降噪	
	AEC	NULL	回声消除	
	ROUTE	1.0.0	通道映射	
	VOLUME	1.0.0	通道音量调节	
	PHASE	1.0.0	通道相位时延调节	
	PASS FILTER	1.0.0	高/低通滤波器	
远程类	EQ	1.1.0	均衡器	√
	ANS	NULL	降噪	

版权所有 © 珠海全志科技股份有限公司。保留一切权利

算法类型	算法名称	算法版本	功能	上位机调参
	AEC	NULL	回声消除	
	PASS FILTER	1.0.0	高/低通滤波器	

- 新增远程类算法 (高/低通滤波器)；
- 支持 EQ 分通道参数设置。

2.3 组件关系

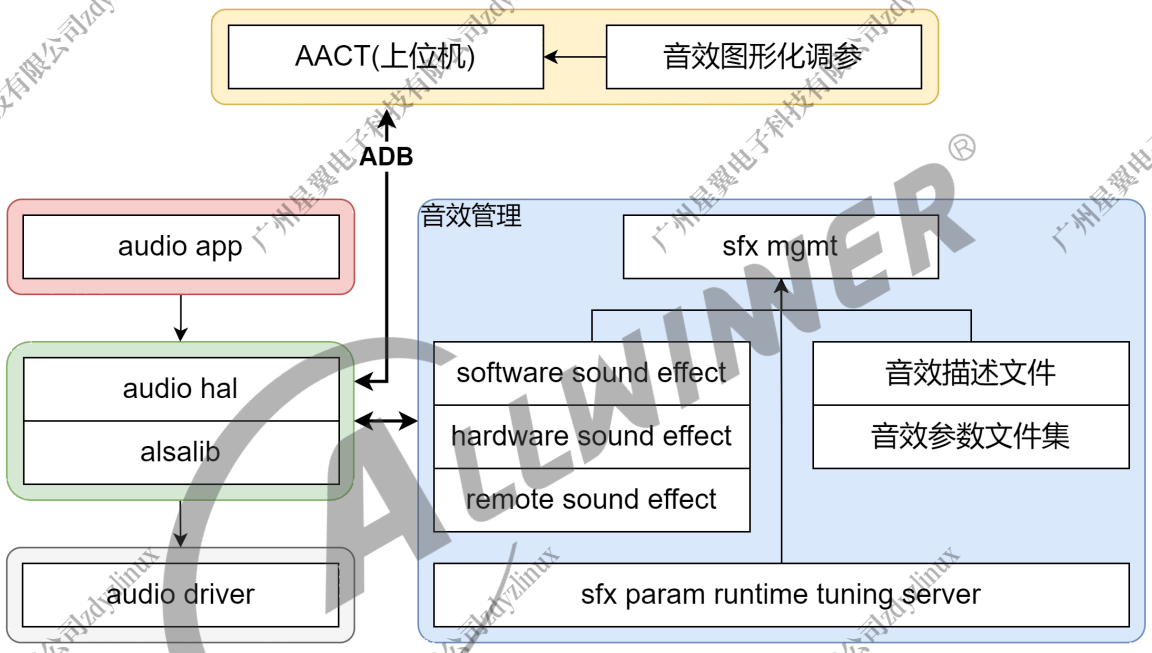

图 2-1: sfx_mgmt 组件关系

音效管理架构在音频系统中的组件关系如图所示，介绍如下。

1、**音效管理：**包含音效描述文件、音效参数配置文件集、软件类音效、硬件类音效、远程处理音效、音效参数实时调节服务，为音频平台提供方便快捷的音效集成和调试服务；

2、**AACT 上位机：** Allwinnertech SoC 的音频调试工具，提供音效图形化调参，结合音效管理架构可在上位机实时调整音效参数，提高音效调优效率；

3、**audio app：**音频应用层，如音乐播放器等，在 Android 系统中调用 audiohal 进行实际播放录音；

4、**audio hal：**音频硬件抽象层，使用 alsa 相关库，管理声卡，音效管理架构集成在此处；

5、**audio driver：**音频驱动层，提供声卡播录的具体实现。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

其中音效管理、AACT 上位机为该文档所介绍部分。

2.4 目录结构

在该音效管理架构已集成入音频系统的基础，从使用上需修改的文件为音效管理部分，该章节仅介绍音效管理架构相关目录结构。

2.4.1 音效管理目录

Android13

```
hardware/aw/audio/sfx_mgmt
.
├── demo
│   ├── plugin
│   │   └── sw_sfx
│   │       ├── sfx_adapter.c
│   │       ├── sfx.c
│   │       ├── sfx.h
│   │       ├── sfx_parser.c
│   │       └── sfx_parser.h
│   └── sfx_mgmt_demo.c
├── libAwSfxMgmt
│   ├── include
│   │   └── sfx_mgmt.h
│   ├── lib32
│   │   └── libAwSfxMgmt.so
│   └── lib64
│       └── libAwSfxMgmt.so
├── librpbuf
│   ├── include
│   │   └── librpbuf.h
│   ├── lib32
│   │   └── librpbuf.so
│   └── lib64
│       └── librpbuf.so
└── sfx_parm_adjust
    ├── arm
    │   └── sfx_parm_adjust
    └── arm64
        └── sfx_parm_adjust

device/softwinner/{chip code}/{board}/media/audio/sfx_mgmt
.
├── audio_device_desc.xml
├── audio_{stream}_{sfx1}_param.xml
├── audio_{stream}_{sfx2}_param.xml
└── ...

1、libAwSfxMgmt.so/.a：音效管理架构库(含算法)；
2、sfx_parm_adjust ：音效管理实时调参服务；
3、sfx_mgmt_demo  ：测试demo；
```

```
4、demo/plugin    ：插件开发demo示例；
5、librpbuf       ：多核异构通信方式。
```

buildroot

```
platform/allwinner/audio/sfx_mgmt
.
├── config
│   ├── audio_device_desc.xml
│   ├── audio_{stream}_{sfx1}_param.xml
│   ├── audio_{stream}_{sfx2}_param.xml
├── demo
│   ├── plugin
│   │   └── sw_sfx
│   │       ├── sfx_adapter.c
│   │       ├── sfx.c
│   │       ├── sfx.h
│   │       ├── sfx_parser.c
│   │       └── sfx_parser.h
│   └── sfx_mgmt_demo.c
├── adjust
│   └── sfx_parm_adjust.c
├── include
│   ├── sfx_list.h
│   ├── sfx_log.h
│   ├── sfx_mgmt.h
│   └── sfx_plugin.h
└── lib
    └── arm64
        ├── libsfx_mgmt.a
        └── libsfx_mgmt.so

1、libsfx_mgmt.so/.a：音效管理架构库(含算法)；
2、sfx_parm_adjust  ：音效管理实时调参服务；
3、sfx_mgmt_demo    ：测试demo；
4、demo/plugin      ：插件开发demo示例。
```

配置文件

- {chip code}: 芯片代号，如 saturn；
- {board}: 具体板型，如 common；
- {stream}: 数据流，有“play” 和 “cap” 可选；
- {sfx}: 音效算法，如 “hw_drc”、“hw_hpf”、“eq”、“AGC”。

1、audio_device_desc.xml

音效设备描述文件，描述各个音效设备使用的音效组合及音效参数。

2、audio_{音频流}_{算法名称}_param.xml

音效参数配置文件，描述所使用音效算法的参数。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4.2 远程音效目录

Rtos

```
rtos-components/aw/sfx_mgmt
.
├── Kconfig
├── Makefile
├── libsfxmgmt_hifi4.a
└── sfx_mgmt.h
```

libsfxmgmt_hifi4.so 远程音效管理库 (含算法)，运行在 DSP 核。

广州星翼电子科技有限公司zdyzlinux

3 模块接口说明

3.1 源文件列表

3.1.1 linux 端源码

音效管理

```
frame/
├── tinyxml
│   ├── tinyxml.c
│   └── tinyxml.h
├── sfx_plugin
│   └── sfx_plugin.c
├── parm_server
│   ├── parm_server.c
│   └── parm_server.h
├── platform
│   ├── sfx_platform.c
│   ├── sfx_platform.h
│   ├── sfx_platform_rp.c
│   └── sfx_platform_rp.h
├── sfx_log.h
├── sfx_list.h
├── sfx_plugin.h
├── sfx_mgmt.h
└── sfx_version.h
```

音效插件

```
frame/
├── hw_effect
│   ├── {hw_sfx}
│   │   ├── {hw_sfx}.c
│   │   ├── {hw_sfx}.h
│   │   ├── {hw_sfx}_parser.c
│   │   ├── {hw_sfx}_parser.h
│   │   └── {hw_sfx}_adapter.c
│   └── ...
├── rp_effect
│   ├── {rp_sfx}
│   │   ├── {rp_sfx}.h
│   │   ├── {rp_sfx}_parser.c
│   │   ├── {rp_sfx}_parser.h
│   │   └── {rp_sfx}_adapter.c
│   └── ...
└── sw_effect
    ├── {sw_sfx}
    │   ├── {sw_sfx}.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    ├──── {sw_sfx}.h
    │   ├──── {sw_sfx}_parser.c
    │   ├──── {sw_sfx}_parser.h
    │   └──── {sw_sfx}_adapter.c
    └──── ...
```

音效适配层源码有 3 种类型，分别为 hw_effect、rp_effect、sw_effect，用于将各种类型算法抽象为统一的音效插件，其结构包含音效算法、参数解析、算法适配。

1、音效算法：算法具体实现，但对于远程音效，算法实现源码在其它核；

2、参数解析：非必须项，若无参数解析需求，可将默认参数固化于源码中；

3、算法适配：必须项，需将算法 api 接口填充至统一的音效算法插件函数。

音效 IO

```
frame/
 ├──── hw_effect
 │   ├──── hw_sfx_io.c
 │   └──── hw_sfx_io.h
 └──── rp_effect
    ├──── rpbuf
    │   ├──── rpsfx_rpbuf.c
    │   └──── rpsfx_rpbuf.h
    ├──── rpsfx_io.c
    └──── rpsfx_io.h
```

1、hwsfx_io：用于寄存器读写，从而控制硬件音效算法；

2、rpsfx_io：用于远程音效读写，从而控制远程音效算法；

3、rpbuf：一种多核异构通信方式的封装，由 rpsfx_io 调用。

支持库

```
supportlibs/
 ├──── librpbuf
 │   └──── ...
 └──── tinyalsa
 │   └──── ...
 └──── ...
```

1、librpbuf：一种多核异构通信方式的实现；

2、tinyalsa：alsa 库，用于声卡操作。

其它

```
adjust/
 └──── parm_adjust.c
config/
 ├──── audio_device_desc.xml
 ├──── audio_{stream}_{sfx}_param.xml
 └──── ...
audio_file/
 ├──── {sfx}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
│   └──── ...
└──── ...
demo/
└──── sfx_mgmt_demo.c
```

1、adjust：实时调参程序，上位机或手动调用该程序，实现调参；

2、config：音效设备描述文件、音效参数配置文件；

3、audio_file：测试音频；

4、demo：测试程序。

3.1.2 rtos 端源码

```
frame/
├──── adapter
│   ├──── sfx_adapter.c
│   └──── sfx_adapter.h
├──── platform
│   ├──── rpsfx.c
│   └──── rpsfx.h
├──── rp_effect
│   ├──── {rp_sfx}
│   │   ├──── {rp_sfx}.c
│   │   ├──── {rp_sfx}.h
│   │   ├──── {rp_sfx}_parser.c
│   │   ├──── {rp_sfx}_parser.h
│   │   └──── {rp_sfx}_adapter.c
│   ├──── ...
│   ├──── rpbuf
│   │   ├──── rpsfx_rpbuf.c
│   │   └──── rpsfx_rpbuf.h
│   ├──── rpsfx_io.c
│   └──── rpsfx_io.h
├──── sfx_log.h
├──── sfx_list.h
├──── sfx_plugin.h
├──── sfx_mgmt.h
├──── sfx_reg_plugin.h
└──── sfx_version.h
```

相对于 linux 端源码，仅保留远程音效和音效管理两部分，同时增加远程音效算法源码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2 软件框图

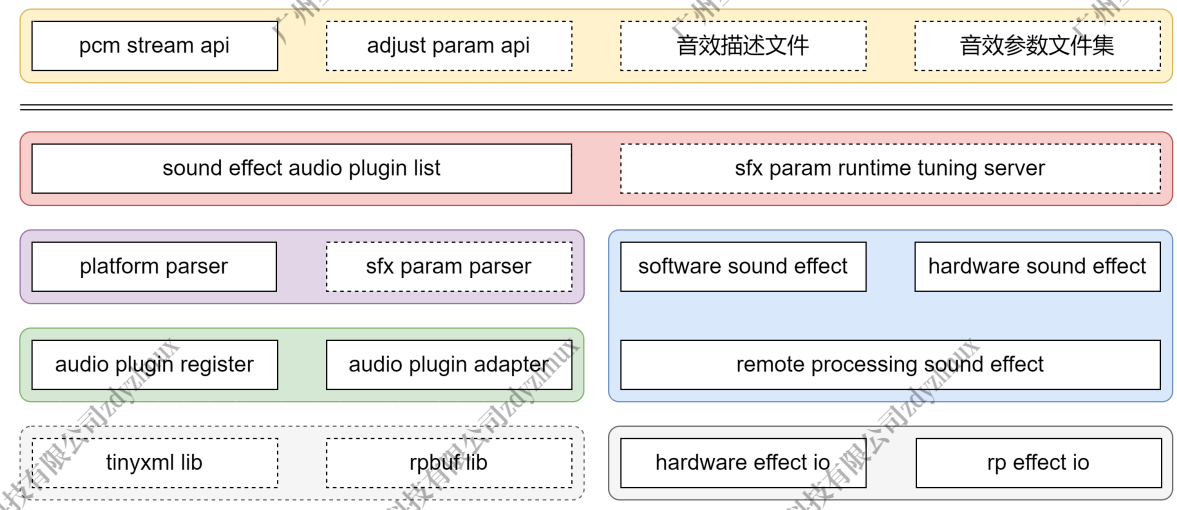

图 3-1: sfx_mgmt 框图

软件框架由以下 7 部分组成。

1、**外部接口：**包含 pcm 流接口、参数实时调参接口、音效设备描述文件、音效参数配置文件。提供统一、透明的应用层接口，同时可通过配置文件实现根据不同音效设备及应用场景使用不同音效组合及音效参数的灵活配置需求；

2、**音效管理：**包含音效插件链、音效实时调参服务。用于调用音效和调节音效参数，是整个架构的核心；

3、**配置解析：**包含音效设备描述文件、音效参数配置文件解析。负责解析相关配置文件，并把解析结果向音效管理层上报；

4、**音效插件：**包含音效插件适配和注册。负责提供统一的音效算法抽象接口，以及向音效管理注册音效插件；

5、**音效：**包含软件类、硬件类、远程处理类音效。提供音效的具体实现，并抽象适配至音效插件；

6、**音效 IO：**包含硬件音效、远程处理音效的 IO 接口。提供硬件音效寄存器读写和多核异构通信读写功能；

7、**支持库：**包含配置文件解析库、多核异构通信库。提供配置文件解析和多核异构通信接口，供配置解析和远程处理音效使用。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 软件主要流程

3.3.1 数据处理流程

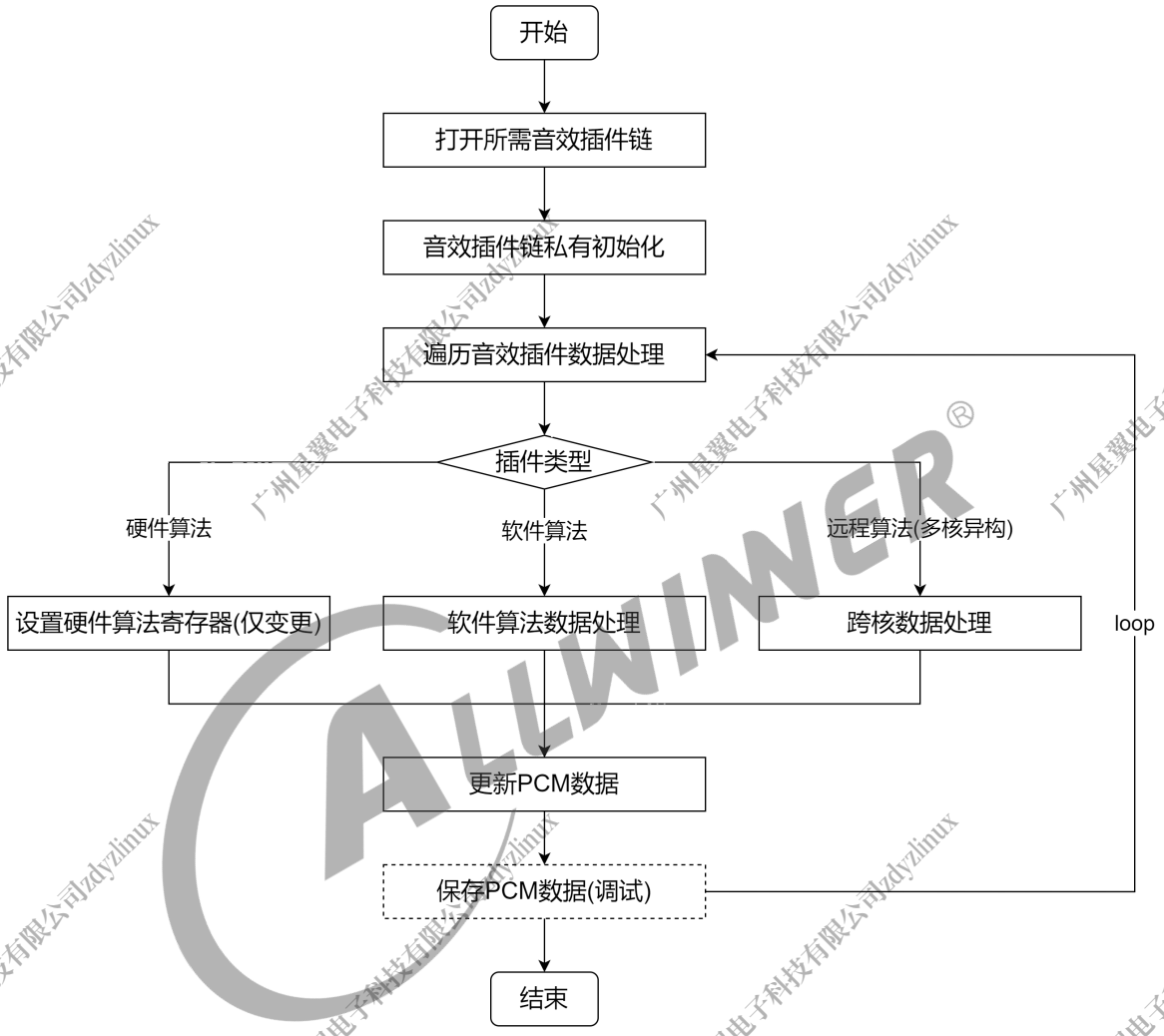

图 3-2: sfx_mgmt 数据处理流程

开始数据处理时，首先根据场景需求打开对应音效插件链，并对该音效插件链下所有音效进行私有初始化。以上处理后进入音效处理循环，将待处理的 pcm 音频数据送入音效插件链，逐一遍历所有音效插件，并根据不同插件类型使用不同音效算法处理，遍历结束后更新 pcm 数据，若需要保存 pcm 数据进行调试，则根据所指定的某一个或多个音效需求，保存其处理后的 pcm 数据为文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.2 远程音效流程

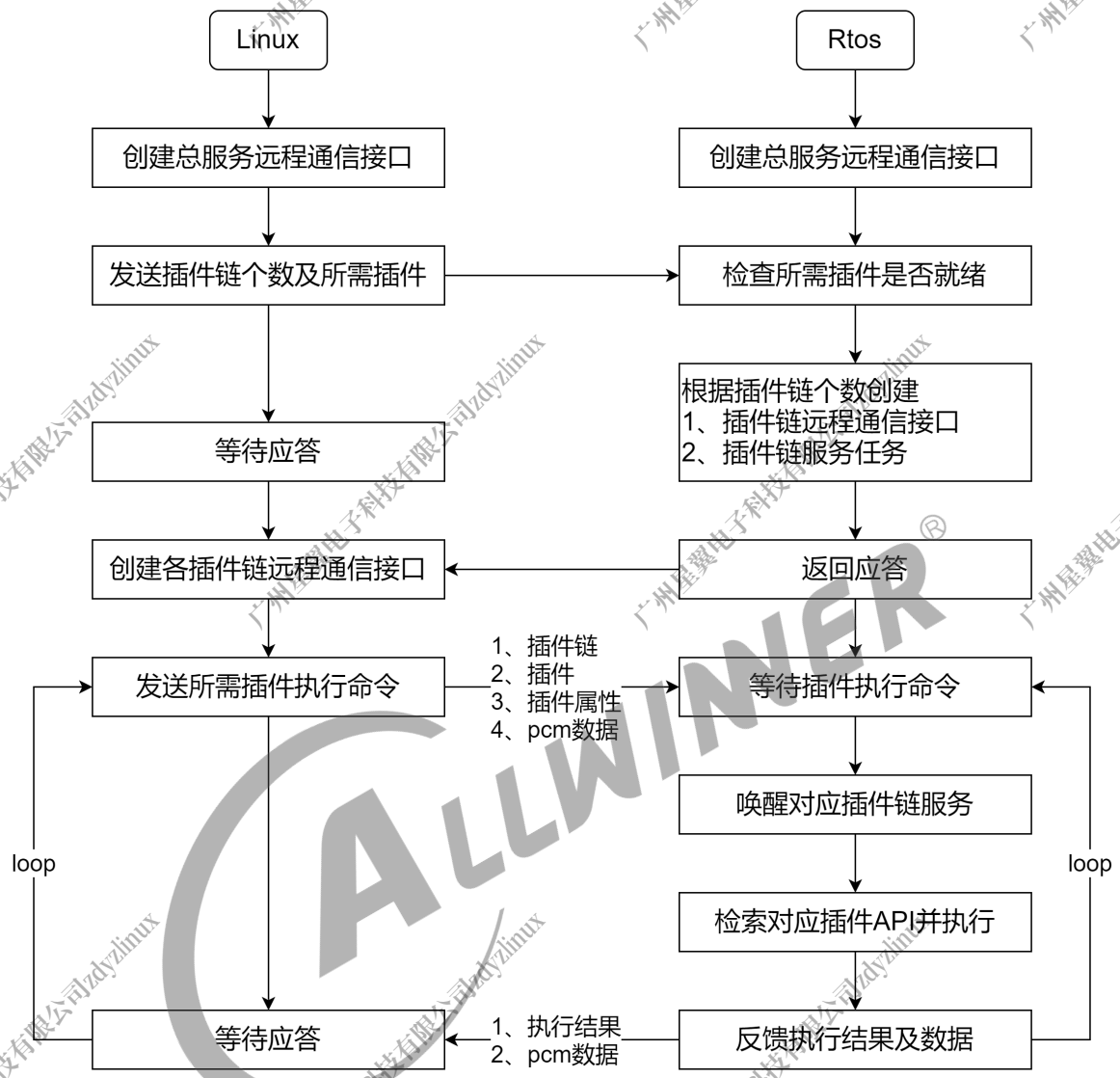

图 3-3: sfx_mgmt 远程音效

远程音效处理流程基于多核异构平台，其中音效使用控制等在 linux 端 (通常为 ARM 核) 实现，音效具体实现在 rtos 端 (通常为 DSP 核) 实现，其流程如下。

1. linux 端和 rtos 端创建音效管理总服务远程通信连接；
2. linux 端发送所需插件链个数和所需插件，并创建对应远程通信接口；
3. rtos 端接收插件链个数和插件，对应创建插件链服务任务、远程通信接口，并反馈结果；
4. linux 端发送插件执行命令，包含插件链、插件、插件属性、pcm 数据等，并进入等待应答；
5. rtos 端接收执行命令，唤醒对应插件链服务任务，根据属性执行对应插件，并反馈结果。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.3 实时调参流程

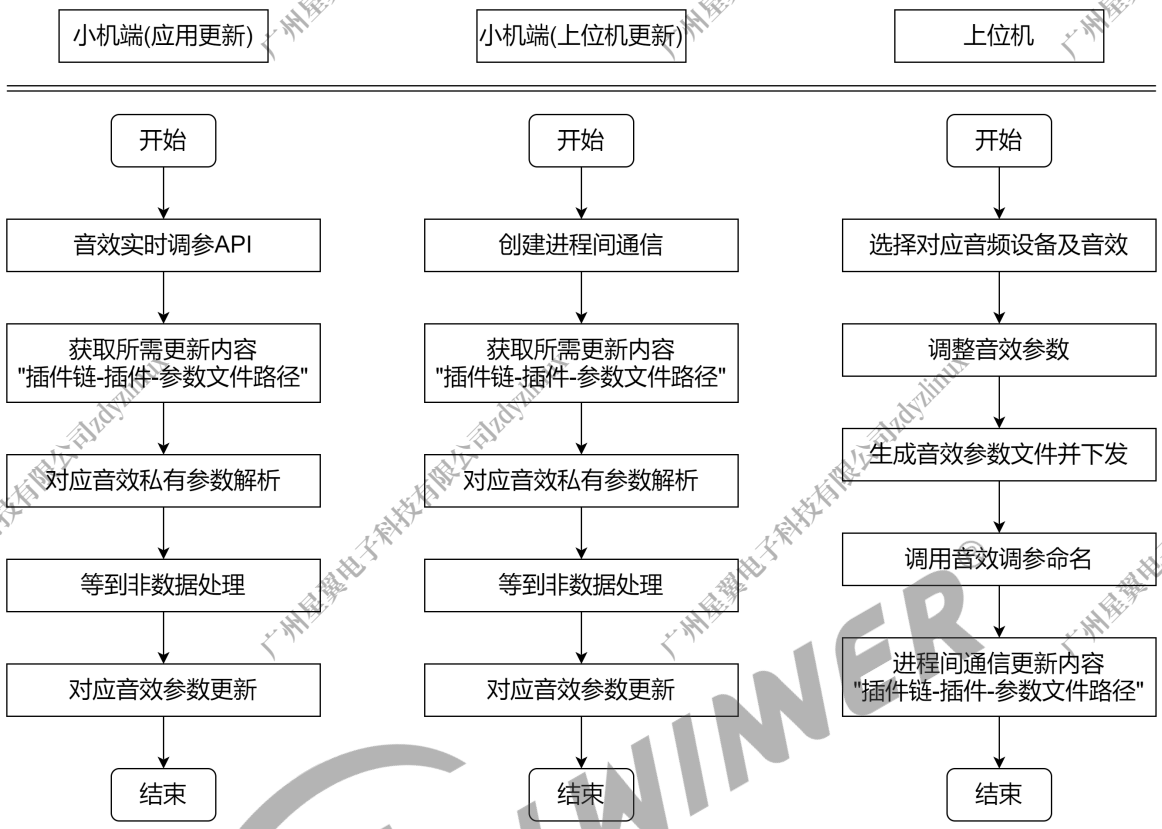

图 3-4: sfx_mgmt 实时调参

实时调参流程分为小机端应用发起调参，和上位机发起调参。

小机端应用发起调参：

sfx_mgmt 提供音效实时调参 API，输入 “插件链-插件-参数文件路径” ，API 内部进行检索对应插件，并调用对应插件音效参数解析接口，等待非 pcm 流数据处理时，进行对应插件音效参数更新，达到实时调参效果。

上位机发起调参

1. 在 sfx_mgmt 提供音效实时调参 API 基础上，小机端提供实时调参服务和实时调参应用程序；
2. 上位机选择音效设备 (映射为音效插件链)，根据所需调整音效参数，并生成音效参数文件；
3. 上位机下发音效参数文件至小机端，并调用实时调参应用程序，达到上位机调参效果。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4 关键数据结构

3.4.1 sfx_platform

```
struct sfx_platform {
    char name[12];
    char board[12];
    unsigned int hwsfx_base;
    unsigned int hwsfx_size;
    char *dump_dirpath;
    bool hw_effect_exist;
    bool rp_effect_exist;
    uint8_t audio_dev_cnt;
    struct sfx_list_head *ap_list;
    struct sfx_audio_device *audio_dev;

    /* register audio plugin */
    struct sfx_plugin *reg_ap;
    unsigned int reg_ap_cnt;

    pthread_rwlock_t param_setup_lock;
};
```

描述音效管理结构体，包含平台信息、硬件音效 IO、音效设备、音效插件链等，用于音效管理创建。

3.4.2 sfx_ap_dev

```
struct sfx_ap_dev {
    struct sfx_list_head *ap_list;
    struct sfx_audio_device *audio_dev;
};
```

音效插件链结构体，包含音效设备、音效插件链，用于音效启动、音效设备数据处理。

3.4.3 sfx_audio_device

```
struct sfx_audio_device {
    int dev_id;
    char dev_name[32];
    char card_name[32];
    enum SFX_STREAM stream;
    uint8_t hw_effect_cnt;
    struct sfx_effect_table *hw_effect_table;
    uint8_t sw_effect_cnt;
    struct sfx_effect_table *sw_effect_table;
    uint8_t rp_effect_cnt;
    struct sfx_effect_table *rp_effect_table;
    struct sfx_pcm_config pcm_config;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    void *priv;
    struct rpsfx_io *rpsfx_io;
};
```

描述音效设备结构体，包含设备类型 (对应插件链)、流方向、pcm 参数、硬件音效表、软件音效表 (包含远程音效) 等，是音效处理的核心结构体。

3.4.4 sfx_plugin

```
/* omit sfx_plugin api element */
struct sfx_plugin {
    const char *ap_name;
    struct sfx_list_head list;
    void *priv;

    /* unchange, shouled be init when defined */
    const enum AP_TYPE type;
    const enum AP_DOMAIN domain;
    const char *param_file_p;
    const char *param_file_c;
    struct sfx_list_head *param_list;

    /* can be changed */
    int dev_id;
    enum AP_STREAM stream;

    /* remoute proccess ap */
    struct rpsfx_io *rpsfx_io;
    uint8_t rpsfx_pool_id;

    enum AP_MODE mode;
    enum AP_PROCESS_SIGN proc_sign;
    struct sfx_pcm_config pcm_config;

    unsigned int dump_last_num;
    unsigned int dump_data_cnt;
    char *dump_file_name;
    FILE *dump_file;
};
```

音效插件结构体，描述插件名称、插件类型、插件处理标志、所属设备、所属处理域等信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.5 外部接口

3.5.1 音效管理接口

3.5.1.1 sfxmgmt_init

- 函数原型：

```
int sfxmgmt_init(struct sfx_platform **platform_info, char *desc_file)
```

- 功能描述：音效管理创建
- 参数说明：
  - platform_info: 音效管理信息
  - desc_file: 音效设备描述文件路径
- 返回值：0-成功，其它-失败

3.5.1.2 sfxmgmt_exit

- 函数原型：

```
void sfxmgmt_exit(struct sfx_platform *platform_info)
```

- 功能描述：音效管理销毁
- 参数说明：
  - platform_info: 音效管理信息
- 返回值：无

3.5.2 数据流接口

3.5.2.1 sfxmgmt_ap_open

- 函数原型：

```
struct sfx_ap_dev *sfxmgmt_ap_open(struct sfx_platform *platform,
                    const char *dev_name,
                    struct sfx_pcm_config *pcm_config)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 功能描述：音效设备启动
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
  - pcm_config: pcm 数据格式
- 返回值：sfx_ap_dev-成功，NULL-失败

3.5.2.2 sfxmgmt_ap_close

- 函数原型：

```
int sfxmgmt_ap_close(struct sfx_ap_dev *ap_dev)
```

- 功能描述：音效设备关闭
- 参数说明：
  - ap_dev: 音效设备信息
- 返回值：0-成功，其它-失败

3.5.2.3 sfxmgmt_ap_do

- 函数原型：

```
int sfxmgmt_ap_do(struct sfx_ap_dev *ap_dev,
        void *data, uint32_t size,
        void **data_out, uint32_t *data_out_size)
```

- 功能描述：音效设备数据处理
- 参数说明：
  - ap_dev: 音效设备信息
  - data: 待处理 pcm 数据
  - size: 待处理 pcm 数据大小
  - data_out: 处理后 pcm 数据
  - data_out_size: 处理后 pcm 数据大小
- 返回值：0-成功，其它-失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.5.3 实时调参接口

3.5.3.1 sfxmgmt_parm_adjust_server

- 函数原型：

```
int sfxmgmt_parm_adjust_server(struct sfx_platform *platform)
```

- 功能描述：实时调参服务，和实时调参应用结合使用
- 参数说明：
  - platform_info: 音效管理信息
- 返回值：0-成功，其它-失败

3.5.3.2 sfxmgmt_parm_setup

- 函数原型：

```
int sfxmgmt_parm_setup(struct sfx_platform *platform, const char *dev_name,
                       const char *ap_name, void *param)
```

- 功能描述：指定某个音效设备的某个音效进行实时调参，非立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
  - ap_name: 音效名称
  - param：音效参数
- 返回值：0-成功，其它-失败

3.5.3.3 sfxmgmt_parm_setup_done

- 函数原型：

```
int sfxmgmt_parm_setup_done(struct sfx_platform *platform,
                            const char *dev_name)
```

- 功能描述：指定某个音效设备的某个音效进行实时调参，立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
- 返回值：0-成功，其它-失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.5.3.4 sfxmgmt_parm_setup_sync

- 函数原型：

```
int sfxmgmt_parm_setup_sync(struct sfx_platform *platform,
                const char *dev_name,
                const char *ap_name, void *param)
```

- 功能描述：指定某个音效设备的某个音效进行实时调参，立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
  - ap_name: 音效名称
  - param：音效参数
- 返回值：0-成功，其它-失败

3.5.3.5 sfxmgmt_parm_setup_byreinit

- 函数原型：

```
int sfxmgmt_parm_setup_byreinit(struct sfx_platform *platform,
                const char *dev_name,
                const char *ap_name, void *param)
```

- 功能描述：指定某个音效设备的某个音效进行实时调参，通过重新初始化方式，非立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
  - ap_name: 音效名称
  - param：音效参数
- 返回值：0-成功，其它-失败

3.5.3.6 sfxmgmt_parm_setup_byreinit_done

- 函数原型：

```
int sfxmgmt_parm_setup_byreinit_done(struct sfx_platform *platform,
                const char *dev_name)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 功能描述：指定某个音效设备的某个音效进行实时调参，通过重新初始化方式，立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
- 返回值：0-成功，其它-失败

3.5.3.7 sfxmgmt_parm_setup_byreinit_sync

- 函数原型：

```
int sfxmgmt_parm_setup_byreinit_sync(struct sfx_platform *platform,
                const char *dev_name,
                const char *ap_name, void *param)
```

- 功能描述：指定某个音效设备的某个音效进行实时调参，通过重新初始化方式，立即生效
- 参数说明：
  - platform_info: 音效管理信息
  - dev_name: 音效设备名称
  - ap_name: 音效名称
  - param：音效参数
- 返回值：0-成功，其它-失败

说明

1. **sfxmgmt_parm_setup 系列函数仅可初始化音效算法所指定参数，可修改参数内容少，但无初始化耗时；**
2. **sfxmgmt_parm_setup_byreinit 系列函数通过重新初始化方式设参，可更改音效算法全部参数，但存在初始化耗时，需结合实际场景是否使用；**
3. **sfxmgmt_parm_setup 和 sfxmgmt_parm_setup_byreinit 函数并非将参数直接生效，而是进行缓存，结合 sfxmgmt_parm_..._done 函数使用，该函数可将缓存的参数实际设置到音效链中，适用于多个音效算法参数需同时修改的情况；**
4. **sfxmgmt_parm_..._sync 函数可将参数直接设置到音效链中，适用于设参不影响其它音效算法的情况。**

3.6 内部接口

3.6.1 音效插件接口 (主机-linux)

3.6.1.1 sfx_plugin adapter

- 函数原型：

```
/* omit sfx_plugin parameter element */
struct sfx_plugin {
    int (*ap_init)(struct sfx_plugin *ap);
    int (*ap_exit)(struct sfx_plugin *ap);
    int (*ap_process)(struct sfx_plugin *ap,
                void *in_data, uint32_t in_size,
                void **out_data, uint32_t *out_size);
    int (*ap_setup)(struct sfx_plugin *ap, void *param);
    int (*ap_setup_byreinit)(struct sfx_plugin *ap, void *param);
    int (*ap_updata_mode)(struct sfx_plugin *ap, enum AP_MODE mode);

    int (*ap_param_parser_all)(struct sfx_list_head *param_list, char *params_file);
    void (*ap_param_free_all)(struct sfx_list_head *param_list);
    int (*ap_param_parser_bydev)(void **param, char *params_file, int dev_id);
    void (*ap_param_free_bydev)(void *param);

    char *(*ap_get_version)(void);
};
```

- ap_init : 音效插件初始化
- ap_exit : 音效插件退出
- ap_process : 音效插件数据处理
- ap_setup : 音效插件参数设置
- ap_setup_byreinit : 音效插件参数设置 (通过重新初始化方式)
- ap_updata_mode : 音效插件模式更新
- ap_param_parser_all : 音效插件全部参数解析
- ap_param_free_all : 音效插件全部参数释放
- ap_param_parser_bydev : 音效插件指定音效设备参数解析
- ap_param_free_bydev : 音效插件指定音效设备参数释放
- ap_get_version : 查看音效管理库版本

3.6.1.2 sfx_plugin register

- 函数原型：

```
#define REGISTER_SFX_MGMT_AP(ap) \
    const audio_plugin_t sfx_mgmt_ap_##ap SECTION("sfx_mgmt_ap") = &(ap)
```

- 功能描述：音效插件注册
- 参数说明：
  - ap: 音效插件结构体
- 返回值：无

3.6.2 音效插件接口 (远程-rtos)

3.6.2.1 sfx_plugin adapter

- 函数原型：

```
/* omit sfx_plugin parameter element */
struct sfx_plugin {
   int (*ap_init)(struct sfx_plugin *ap, void *param);
   int (*ap_exit)(struct sfx_plugin *ap);
   int (*ap_process)(struct sfx_plugin *ap,
              void *in_data, uint32_t in_size,
              void **out_data, uint32_t *out_size);
   void (*ap_setup)(struct sfx_plugin *ap, void *param);
};
```

- ap_init : 音效插件初始化
- ap_exit : 音效插件退出
- ap_process : 音效插件数据处理
- ap_setup : 音效插件参数设置

3.6.3 硬件音效 IO 接口

说明

硬件音效 IO 需驱动将硬件音效寄存器域映射至用户态才可使用。

3.6.3.1 hw_sfx_read

- 函数原型：

```
int hw_sfx_read(struct hw_sfx_io *hw_sfx_io, unsigned int reg, unsigned int *val)
```

- 功能描述：硬件音效寄存器读
- 参数说明：
  - hw_sfx_io: 硬件音效 IO
  - reg: 寄存器地址
  - val: 寄存器地址返回值
- 返回值：0-成功，其它-失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.6.3.2 hw_sfx_write

- 函数原型：

```
int hw_sfx_write(struct hw_sfx_io *hw_sfx_io, unsigned int reg, unsigned int val)
```

- 功能描述：硬件音效寄存器写
- 参数说明：
  - hw_sfx_io: 硬件音效 IO
  - reg: 寄存器地址
  - val: 寄存器地址值
- 返回值：0-成功，其它-失败

3.6.3.3 hw_sfx_update_bits

- 函数原型：

```
int hw_sfx_update_bits(struct hw_sfx_io *hw_sfx_io, unsigned int reg,
                       unsigned int mask, unsigned int val)
```

- 功能描述：硬件音效寄存器写 (带掩码)
- 参数说明：
  - hw_sfx_io: 硬件音效 IO
  - reg: 寄存器地址
  - mask: 寄存器掩码
  - val: 寄存器地址值
- 返回值：0-成功，其它-失败

3.6.4 远程音效 IO 接口

3.6.4.1 rpsfx io adapter

- 函数原型：

```
/* omit rpsfx_io parameter element */
struct rpsfx_io {
  int (*rpsfx_init)(struct rpsfx_io *rp);
  int (*rpsfx_exit)(struct rpsfx_io *rp);
  int (*rpsfx_send)(struct rpsfx_io *rp, struct rpsfx_data *data);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    int (*rpsfx_recv)(struct rpsfx_io *rp, struct rpsfx_data *data);
};
```

- rpsfx_init : 远程音效 IO 初始化
- rpsfx_exit : 远程音效 IO 销毁
- rpsfx_send : 远程音效 IO 数据发送
- rpsfx_recv : 远程音效 IO 数据接收

3.7 调试接口

3.7.1 文件保存

> TODO: 目前通过配置文件实现音效处理后数据文件保存，后续将新增 api 动态使用文件保存功能。

通过 audio_device_desc.xml 的 dump_cnt 属性配置，支持指定任意算法多文件循环保存。

3.7.2 耗时调试

> TODO: 待开发。

广州星翼电子科技有限公司zdyzlinux

4 应用使用方法

提供至应用层使用共两类接口，一类为 pcm 流处理接口，一类为实时调参接口，其中 pcm 流处理接口对所使用音效算法透明处理，使用者无需关注内部算法细节。共提供以下三种使用方式。

4.1 依赖环境配置

仅使用软件算法

1. 获取音效管理库；
2. audio_device_desc.xml 文件中不可配置 hw_effect 和 rp_effect。

需使用硬件算法，不使用远程算法

1. 获取音效管理库；
2. audio_device_desc.xml 文件中不可配置 rp_effect；
3. 内核 menuconfig 配置勾选 SND_SOC_SUNXI_SFX，并确认生成/dev/sfx_mgmt 节点。

需使用远程算法 (rpbuf 通信方式)，不使用硬件算法

1. 获取音效管理库；
2. audio_device_desc.xml 文件中不可配置 hw_effect；
3. 内核 menuconfig 配置选择 rpmsg、rpbuf 相关配置，具体查看《RPBuf 使用指南》；
4. dsp 核正常启动，且 dsp 固件中包含远程音效管理库。

需同时使用硬件算法和远程算法 (rpbuf 通信方式)

1. 获取音效管理库；
2. 内核 menuconfig 配置勾选 SND_SOC_SUNXI_SFX，并确认生成/dev/sfx_mgmt 节点；
3. 内核 menuconfig 配置选择 rpmsg、rpbuf 相关配置，具体查看《RPBuf 使用指南》；
4. dsp 核正常启动，且 dsp 固件中包含远程音效管理库。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 直接使用

1. 音效管理创建，用于解析创建各音效链，音频相关应用启动时调用；
2. 音效管理销毁，用于销毁所有音效管理架构相关内容，在音频相关应用关闭时调用；
3. 音效设备启动，指定所需音效链，返回音效设备，在音频相关应用起播或起录调用；
4. 音效设备关闭，指定音效设备，完成对应音效链销毁，在音频相关应用停播或停录调用；
5. 音效设备数据处理，指定音效设备，完成对应音效链数据处理，在音频相关应用播或录时调用；
6. 音效实时调参，指定音效链和音效，以及新参数文件路径，完成参数更新，在音频相关应用需变更音效参数时调用。

4.3 集成 tinyalsa 使用

针对使用 tinyalsa 库且不直接使用 sfx_mgmt 相关接口的情况，可采用将 sfx_mgmt 外部接口封装入 tinyalsa 库的方式使用，从而不影响上层应用已有的应用代码结构，具体方式如下。

1. 将 “音效管理创建” 和 “音效设备启动” 封装入 pcm_open 函数中，通过指定 pcm 设备名称调用；
2. 将 “音效管理销毁” 和 “音效设备关闭” 封装入 pcm_close 函数中，通过指定 pcm handle 调用；
3. 将 “音效设备数据处理” 封装入 pcm_read 和 pcm_write 函数中，通过指定 pcm handle 调用；
4. 将 “音效实时调参” 封装入 mixer 相关操作函数中，通过 mixer 控件操作调用。

4.4 集成 alsalib 使用

针对使用 alsalib 库且不直接使用 sfx_mgmt 相关接口的情况，可采用将 sfx_mgmt 外部接口封装入 alsalib 库的方式使用，从而不影响上层应用已有的应用代码结构，具体方式如下。

1. 将 “音效管理创建” 和 “音效设备启动” 封装入 snd_pcm_open 函数中，通过指定 pcm 设备名称调用；
2. 将 “音效设备启动” 和 “音效设备关闭” 封装入 snd_pcm_close 函数中，通过指定 pcm handle 调用；
3. 将 “音效设备数据处理” 封装入 snd_pcm_read 和 snd_pcm_write 函数中，通过指定 pcm handle 调用；
4. 将 “音效实时调参” 封装入 mixer 相关操作函数中，通过 mixer 控件操作调用。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.5 外部接口使用示例

pcm 流处理

```
/* sfx mgmt init */
struct sfx_platform *platform = NULL;
sfxmgmt_init(&platform, SFX_MGMT_AUDIO_DEVICE_DESC);

void demo_func_play()
{
  /* open sound card */
  pcm = pcm_open(card, device, PCM_OUT, &pcm_config);

  /* sfx dev open */
  sfx_pcm_config.channels = pcm_config.channels;
  sfx_pcm_config.rate = pcm_config.rate;
  sfx_pcm_config.bit = 16;
  sfx_pcm_config.format = SFX_PCM_FORMAT_S16_LE;
  sfx_pcm_config.period_size = pcm_config.period_size;
  sfx_pcm_config.period_count = pcm_config.period_count;
  ap_dev = sfxmgmt_ap_open(platform, dev_name, &sfx_pcm_config);

  /* sfx dev process */
  do {
    sfxmgmt_ap_do(ap_dev, in, in_size, &out_data, &out_size);
    pcm_write(pcm, out_data, out_size);
  } while (!close);

  /* sfx dev close */
  sfxmgmt_ap_close(ap_dev);

  /* close sound card */
  pcm_close(pcm);
}

/* sfx mgmt exit */
sfxmgmt_exit(platform);
```

说明

1. **sfxmgmt_init 函数在整个进程中调用一次即可，sfxmgmt_ap_open 按需调用；**
2. **对于播放流，先执行 sfxmgmt_ap_do ，再执行 pcm_write；**
3. **对于录音流，先执行 pcm_read ，再执行 sfxmgmt_ap_do。**

4.6 配置文件使用

4.6.1 音效设备描述文件

文件命名

```
audio_device_desc.xml
```

4.6.1.1 文件格式说明

表 4-1: 音效设备描述文件格式参数

key-val	解释说明
audio_device_desc->version	该音效设备描述文件版本
platform_info	板型平台说明
platform_info->platform	芯片名称、板型、内核版本
platform_info->hwsfx	硬件音效可设置偏移及范围
platform_info->dump	音效处理后文件保存路径
platform_devices	平台设备
platform_devices->play_device	播放流音效设备
platform_devices->play_device->type	播放流音效设备类型
platform_devices->play_device->audiocodec	该播放流音效设备所使用声卡
platform_devices->play_device->sw_effect	该播放流音效设备所要支持的软件音效
platform_devices->play_device->hw_effect	该播放流音效设备所要支持的硬件音效
platform_devices->play_device->rp_effect	该播放流音效设备所要支持的远程音效
platform_devices->cap_device	录音流音效设备
platform_devices->cap_device->type	录音流音效类型
platform_devices->cap_device->audiocodec	该录音流音效所使用声卡
platform_devices->cap_device->sw_effect	该录音流音效所要支持的软件音效
platform_devices->cap_device->hw_effect	该录音流音效所要支持的硬件音效
platform_devices->cap_device->rp_effect	该播放流音效设备所要支持的远程音效

play_device 和 cap_device

数量根据需求而定，允许 0 个、1 个、多个。

play_device->type

可选值支持如下 (根据需求新增)。

- Speaker - 喇叭
- Hpout - 耳机

cap_device->type

可选值支持如下 (根据需求新增)。

- MIC - 麦克风
- HMIC - 耳机麦克风

版权所有 © 珠海全志科技股份有限公司。保留一切权利

sw_effect

不限定播放设备或录音设备，可选值支持如下 (根据需求新增)。

- null - 不使用软件算法
- EQ
- AGC

hw_effect

不同芯片平台有所差异，可选值支持如下 (根据需求新增)。

⚠ 注意

1、播放流音效设备，hw_effect 类音效需放置于 sw_effect 后；
2、录音流音效设备，sw_effect 类音效需放置于 hw_effect 后。

4.6.1.2 文件示例

```
<?xml version="1.0" encoding="UTF-8"?>
<audio_device_desc version="0.1">
    <!-- test: audio device desc -->
    <platform_info>
        <platform name="A523" board="pro3" kernel_version="linux5.15"/>
        <hwsfx base="0xF0" size="0x2BC"/>
        <dump dirpath="./out/"/>
    </platform_info>

    <platform_devices>
        <play_device type="Speaker" card_name="audiocodec">
            <sw_effect sw="AGC" dump_cnt="2"/>
            <sw_effect sw="EQ" dump_cnt="3"/>
            <sw_effect sw="ROUTE"/>
            <hw_effect hw="HW-DRC"/>
        </play_device>
        <play_device type="Hpout" card_name="audiocodec">
            <!-- sw_effect sw="xxx" -->
            <hw_effect hw="HW-DRC"/>
            <hw_effect hw="HW-HPF"/>
        </play_device>
        <cap_device type="MIC" card_name="audiocodec">
            <hw_effect hw="HW-HPF"/>
            <sw_effect sw="AGC"/>
            <sw_effect sw="EQ"/>
        </cap_device>
        <cap_device type="HMIC" card_name="audiocodec">
            <!-- hw_effect hw="xxx" -->
            <!-- sw_effect sw="xxx" -->
        </cap_device>
    </platform_devices>
</audio_device_desc>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

该示例表示有 2 个播放流音效设备和 2 个录音流音效设备，如下。

播放流音效设备 1：

- 设备类型：Speaker (扬声器)
- 软件类算法：AGC, EQ, ROUTE (处理顺序为 AGC->EQ->ROUTE)
- 硬件类算法：HW-DRC
- 处理后文件保存：AGC 处理后文件循环保存，最多存在 2 个文件
- 处理后文件保存：EQ 处理后文件循环保存，最多存在 3 个文件

播放流音效设备 2：

- 设备类型：Hpout (耳机)
- 软件类算法：不使用
- 硬件类算法：HW-DRC, HW-HPF

录音流音效设备 1：

- 设备类型：MIC (麦克风)
- 硬件类算法：HW-HPF
- 软件类算法：EQ，AGC (处理顺序为 EQ->AGC)

录音流音效设备 2：

- 设备类型：HMIC (耳机麦)
- 硬件类算法：不使用
- 软件类算法：不使用

4.6.1.3 文件修改示例

修改前：

```
<play_device type="Speaker" card_name="audiocodec">
    <sw_effect sw="EQ"/>
    <hw_effect hw="HW-DRC"/>
</play_device>
```

修改后：

```
<play_device type="Speaker" card_name="audiocodec">
    <sw_effect sw="EQ"/>
    <sw_effect sw="AGC"/>
    <hw_effect hw="HW-DRC"/>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
</play_device>
```

表示为 “Speaker” 播放流音效设备新增 AGC 软件音效算法，音效执行顺序在 EQ 软件音效算法后。

⚠ 注意

音效描述文件的修改涉及到音效管理对音效设备的创建，若更改该文件，需重启音频服务，或重启机器后生效。

4.6.2 音效参数配置文件

音效参数配置文件使用 xml 格式，有以下要求，分别为文件命名、文件版本、参数树、参数单元记录 (仅用于硬件算法)、参数单元池。

文件命名

```
audio_{stream}_{sfx}_param.xml

e.g.
audio_play_hw_drc_param.xml
audio_cap_hw_hpf_param.xml
audio_play_eq_param.xml
```

4.6.2.1 文件格式说明

文件内容格式 (硬件类音效算法)

```
<?xml version="1.0" encoding="UTF-8"?>
<audio_play_hw_drc_param version="0.1">
  <param_tree>
    <param path="Speaker" param_id="0" enable="1" />
    <param path="Hpout" param_id="1" enable="1" />
    <.../ >
  </param_tree>
  <param_unit_record>
    <param_unit param_id="0">
      <.../ >
      <.../ >
    </param_unit>
    <param_unit param_id="1">
      <.../ >
      <.../ >
    </param_unit>
  </param_unit_record>
  <param_unit_pool>
    <param_unit param_id="0">
      <.../ >
      <.../ >
    </param_unit>
    <param_unit param_id="1">
      <.../ >
```

```
    <.../ >
    </param_unit>
    <.../ >
  </param_unit_pool>
</audio_play_hw_drc_param>
```

文件内容格式 (软件类 & 远程处理类音效算法)

```
<?xml version="1.0" encoding="UTF-8"?>
<audio_play_eq_param version="0.1">
  <param_tree>
    <param path="Speaker" param_id="0" enable="1" />
    <param path="Hpout" param_id="1" enable="1" />
    <.../ >
  </param_tree>
  <param_unit_pool>
    <param_unit param_id="0">
      <.../ >
      <.../ >
    </param_unit>
    <param_unit param_id="1">
      <.../ >
      <.../ >
    </param_unit>
    <.../ >
  </param_unit_pool>
</audio_play_eq_param>
```

1、根节点：

以文件名为根节点。

2、参数树 param_tree：

参数树描述参数 path，和参数 id，以及是否 enable。其中参数 path 对应音效设备描述文件的播放设备类型或录音设备类型，参数 id 和下方的参数单元池对应，enable 和上位机 top 界面的 enable 或 pass 对应。

3、参数单元记录 param_unit_record：(仅用于硬件算法)

由于硬件算法参数基本是寄存器，由算法参数意义计算推到而出，故增加参数单元记录，便于查看算法参数意义，而不用从寄存器反推算法参数意义。

4、参数单元池 param_unit_pool：

描述该算法的对应的播放设备类型或录音设备类型的参数。

4.6.2.2 文件示例

> 以播放流软件 EQ 音效算法为例，即 audio_play_eq_param.xml 文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```xml
<?xml version="1.0" encoding="UTF-8"?>
<audio_play_eq_param version="0.1">
  <!-- test: sw_effect eq -->
  <param_tree>
    <param path="Speaker" param_id="0" enable="1" />
    <param path="Hpout" param_id="1" enable="1" />
  </param_tree>
  <param_unit_pool>
    <param_unit param_id="0">
      <param name="channels" value="2" />
      <param name="enabled" value="1" />
      <param name="bin_num" value="2" />
      <param name="samplerate" value="48000" />
      <!-- filtertype fc gain quailty enable -->
      <param name="BQ1" value="1,242,1,0.6,1" />
      <param name="BQ2" value="1,1876,1,0.6,1" />
    </param_unit>
    <param_unit param_id="1">
      <param name="channels" value="2" />
      <param name="enabled" value="1" />
      <param name="bin_num" value="2" />
      <param name="samplerate" value="48000" />
      <!-- filtertype fc gain quailty enable -->
      <param name="BQ1" value="1,242,1,0.6,1" />
      <param name="BQ2" value="1,1876,1,0.6,1" />
    </param_unit>
  </param_unit_pool>
</audio_play_eq_param>
```

该示例表示有 2 个播放流音效设备的 EQ 参数，如下。

播放流软件 EQ 音效算法参数 1：

- 对应音效设备：Speaker (扬声器)
- 对应参数池 ID：0
- 音效开关：开
- 参数池：

播放流软件 EQ 音效算法参数 2：

- 对应音效设备：Hpout (耳机)
- 对应参数池 ID：1
- 音效开关：开
- 参数池：

说明

修改上述配置文件后的固件位置为 “/vendor/etc/sfx_mgmt”。

4.7 测试 demo

```
./sfx_mgmt_demo
Usage: ./sfx_mgmt_demo {-F function} [Opt ...]
Opt    Val                    note
-F     data                   demo func: audio data test only
       play                   demo func: audio data test with play
       version                demo func: get sfxmgmt and effect Verison
-DEV                          sfx dev
-i     xxx.wav                input wav format audio file
-o     xxx.pcm                output pcm format audio file
-D     card num               sound card number
-d     card device num        sound card device number
-p     period size = #        Interrupt interval # microseconds
-n     period count = #       # period count
-t     sleep time = #         sleep #ms per ap do(only apply data func)
```

测试 demo 提供 3 类功能，分别为数据处理、带声卡播放数据处理、查看版本号，使用方式如上。

4.8 调参 demo

```
./sfx_parm_adjust
Usage: ./sfx_parm_adjust {-DEV dev_type} {-PARM parm_type} {-FILE parm_file}

# -DEV : 指定音效设备，为音效设备描述文件中的设备名称之一；
# -PARM：指定音效算法，为音效设备描述文件中的音效名称之一；
# -FILE：指定音效参数文件。
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 插件开发(基于音效管理库)

5.1 软件类音效插件开发

步骤 1：算法源码获取

获取所需音效算法源码，正常情况下音效算法源码有 init、exit、process 三类 api 接口，另外音效算法参数设置可能为独立接口，也可能集成在 init 接口中。

步骤 2：音效管理库获取

准备音效管理库及其头文件，如下。

```
lib/
 ├──── libsfx_mgmt.a
 └──── libsfx_mgmt.so
include/
 ├──── sfx_list.h     #非必须项
 ├──── sfx_log.h      #非必须项
 ├──── sfx_mgmt.h
 └──── sfx_plugin.h
```

其中 sfx_list.h 提供双链表实现，可使用系统自带双链表实现，非必须使用。sfx_log.h 提供音效管理库统一的打印格式，可使用该文件中的 log 接口，便利于 log 筛查，非必须使用。音效管理库有动态库和静态库两种，按需选择一种即可。

步骤 3：插件适配

创建 {sw_sfx}_adapter.c 文件，在该文件中填充实现 sfx_plugin 接口，如下。

```
struct sfx_plugin ap_sw_sfx = {
    .ap_name        = "SW_SFX_DEMO",
    .ap_init        = ap_sfx_init,
    .ap_exit        = ap_sfx_exit,
    .ap_process     = ap_sfx_process,

    .ap_setup           = ap_sfx_setup,            /* 非必须项 */
    .ap_setup_byreinit  = ap_sfx_setup_byreinit,   /* 非必须项 */
    .ap_updata_mode     = ap_sfx_updata_mode,      /* 非必须项 */
    .ap_param_parser_all = ap_sfx_param_parser_all, /* 非必须项 */
    .ap_param_free_all  = ap_sfx_param_free_all,   /* 非必须项 */
    .ap_param_parser_bydev = ap_sfx_param_parser_bydev, /* 非必须项 */
    .ap_param_free_bydev = ap_sfx_param_free_bydev, /* 非必须项 */
    .ap_get_version     = ap_sfx_version,          /* 非必须项 */

    .type       = AP_TYPE_SW,
    .domain     = AP_DOMAIN_SYS,
    .param_list = &sfx_params_list,                /* 非必须项 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  .param_file_p  = SFX_MGMT_AUDIO_PLAY_SFX_PARAM,     /* 非必须项 */
  .param_file_c  = SFX_MGMT_AUDIO_CAP_SFX_PARAM,      /* 非必须项 */
};
REGISTER_SFX_MGMT_AP(ap_sw_sfx);
```

1、必须实现的接口如下。

ap_name: 该值为字符串，描述音效算法插件名称，用于 audio_device_desc.xml 文件描述某个音效设备所使用的音效名称；

ap_init: 封装所使用音效算法初始化相关内容；

ap_exit: 封装所使用音效算法销毁相关内容；

ap_process: 封装所使用音效算法数据处理相关内容；

type: 值为 AP_TYPE_SW，指定该算法为软件类音效；

domain: 值为 AP_DOMAIN_SYS，指定该算法在系统域运行，即非远程类音效。

2、非必须实现的接口如下。

ap_setup: 封装所使用音效算法独立调参接口相关内容；

ap_setup_byreinit: 通过重新初始化的方式对所使用音效算法进行调参；

ap_sfx_param_parser_all: 解析全部音效算法参数，为所有音效设备所使用的参数；

ap_param_parser_bydev: 解析指定音效设备所使用的参数；

param_list: 参数链表，将 ap_sfx_param_parser_all 解析的所有参数保存在该链表中；

param_file_p: 播放流算法参数文件名称；

param_file_c: 录音流算法参数文件名称；

ap_updata_mode: 更新模式，是否使用该算法的子开关，即 audio_device_desc.xml 文件指定使用后，可通过该接口将其关闭；

ap_get_version: 获取所使用算法版本号。

⚠ 注意

若所使用算法的 api 接口不支持重入，如算法内部使用全局变量，多次 init 将影响其它已 init 的情况，则无法同时给多个音效设备使用，可通过优化算法本身解决，或通过拷贝多份算法规避解决。

说明

非必须接口主要提供参数解析、参数设置相关内容，按需实现即可，如参数解析接口并非一定解析 param_file 文件，也可将所需参数固化在解析接口中。

3、插件注册

使用该宏 REGISTER_SFX_MGMT_AP 将所实现的 sfx_plugin 结构体输入即可。

步骤 4：编译

> 以下仅提供 Makefile 编译示例。

1、静态库

- 将算法源码及步骤 3 中的算法适配源码单独编译为静态库；
- 将新增插件静态库和音效管理静态库统一链接到可执行文件。

示例如下。

```
CFLAGS  := -I ./include
OBJS    += sw_sfx/sfx.o
OBJS    += sw_sfx/sfx_adapter.o

.PHONY: all
all: libsw_sfx_demo.a sfx_mgmt_demo-static

%.o: %.c
   $(CC) -c -o $@ $< $(CFLAGS)

libsw_sfx_demo.a: $(OBJS)
   $(AR) $(ARFLAGS) $@ $^

sfx_mgmt_demo-static: sfx_mgmt_demo.o lib/libsfx_mgmt.a libsw_sfx_demo.a
   $(CC) -Wl,--whole-archive $^ -lm -lpthread -o $@ -Wl,-no-whole-archive
```

2、动态库

- 将算法源码及步骤 3 中的算法适配源码单独编译为静态库；
- 将新增插件静态库和音效管理静态库统一编译链接为动态库；
- 将生成的新动态库连接到可执行文件。

示例如下。

```
CFLAGS  := -I ./include
OBJS    += sw_sfx/sfx.o
OBJS    += sw_sfx/sfx_adapter.o

.PHONY: all
all: libsfx_mgmt_all.so sfx_mgmt_demo-dynamic

%.o: %.c
   $(CC) -c -o $@ $< $(CFLAGS)

libsw_sfx_demo.a: $(OBJS)
   $(AR) $(ARFLAGS) $@ $^

libsfx_mgmt_all.so: lib/libsfx_mgmt.a libsw_sfx_demo.a
   $(CC) $(LDFLAGS) -shared -Wl,-soname,$@ \
   -Wl,--whole-archive $^ -Wl,-no-whole-archive \
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  -o $@

sfx_mgmt_demo-dynamic: sfx_mgmt_demo.o libsfx_mgmt_all.a
  $(CC) $^ -lm -lpthread -o $@
```

5.2 硬件类音效插件开发

和软件类音效插件开发基本一致，区别在于硬件类音效插件增加 hw_io 接口，该接口通过/dev/sfx_mgmt 节点对指定寄存器读写，从而达到配置硬件算法的目的。

说明

/dev/sfx_mgmt 节点仅将硬件音效寄存器域相关内存映射，开发硬件类音效时，需要对应的 spec。

5.3 远程类音效插件开发

TODO: 待开发。

广州星翼电子科技有限公司zdyzlinux

6 上位机实时调参

6.1 前置条件

硬件设备准备

1. 支持 framework net 3.5 以上的 PC；
2. 集成音效管理的板卡；
3. USB 线，并将 PC 和板卡通过 USB 连接。

软件环境准备

1. PC 安装 AACT 工具，工具版本 1.2.0 及以上；
2. 板卡打开 USB adb 调试；
3. 对于播放音效测试，需将对应测试音频存放至板卡。

其它

板卡系统为 android 系统。

- 确保固件版本可进入 root 模式，如 userdebug 固件；
- 确保当前处于开发者模式，同时打开 USB 调试；
- 输入 “**setenforce 0**” 命令，开启文件读取权限。

6.2 音效算法调参

音效算法调参包含音效开关、图形化调参、下发参数三部分，具体调参操作步骤如下。

调参操作步骤简述

1. 启动 AACT 上位机并进入首页界面，工具将自动与设备进行连接，若连接成功 AACT 上位机底部将显示连接状态显示为 Connect，如下图所示；
2. 连接成功后，按钮 “xml from device” 处于激活状态，点击 “xml from device” 按钮从板卡端获取音效配置信息，此时界面上的每个模块按钮会根据配置信息进行相应激活使能，如果未在配置信息中进行配置的模块，则不会使能；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. 选择所需数据流、音效设备，点击所需音效算法图标，即可进入所需音效调参界面；
4. 音效调参界面中进行图形化调参、下发参数至板卡，完成调参。

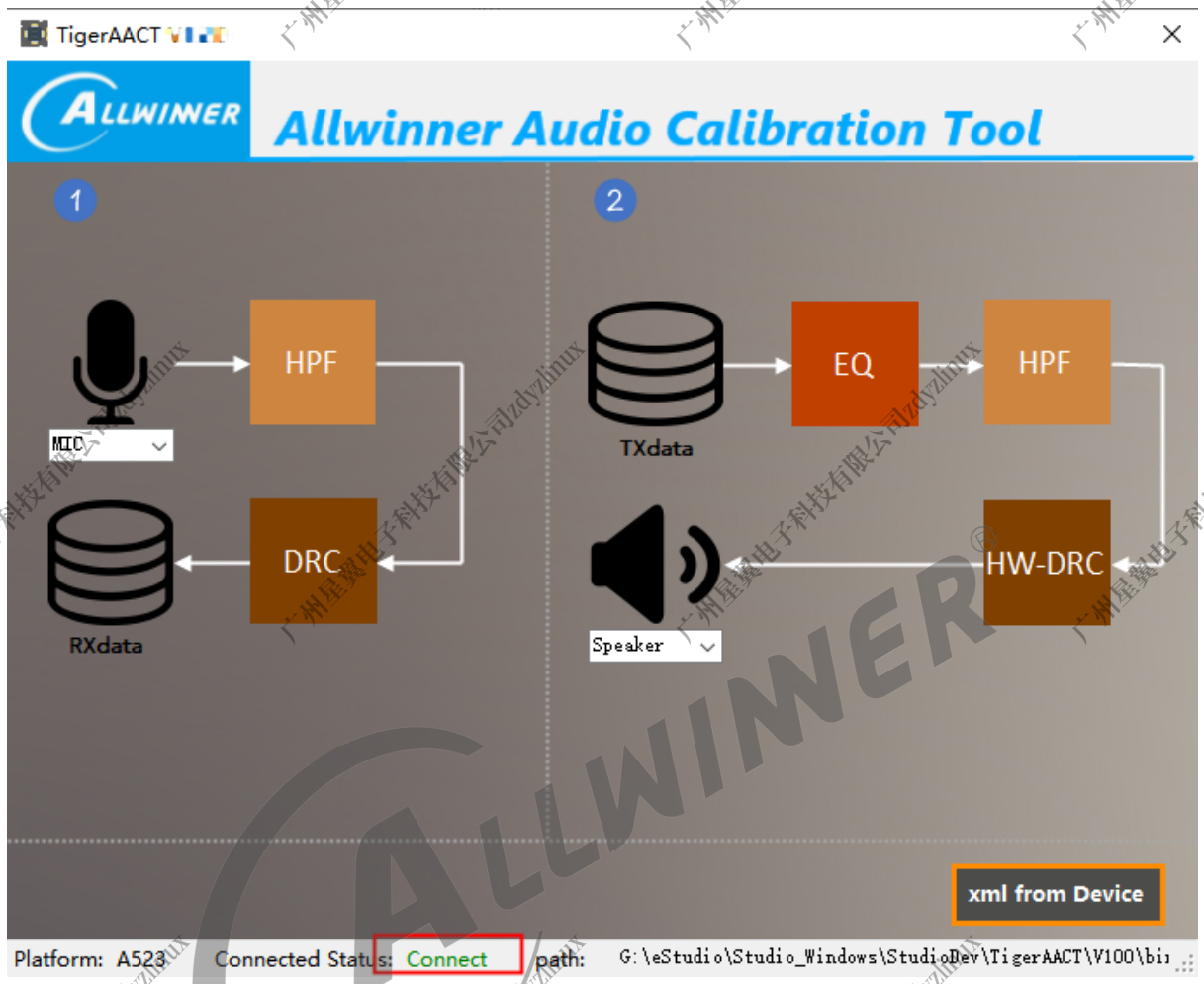

图 6-1: tigerAACT 界面-连接

说明

1. **图形化调参见图形化调参 章节；**
2. **下发参数至板卡见下发参数 章节。**

6.2.1 图形化调参

AACT 工具为音效管理架构支持的音效算法创建图形化调参界面，以 HW-HPF 音效算法为例，如下图所示。

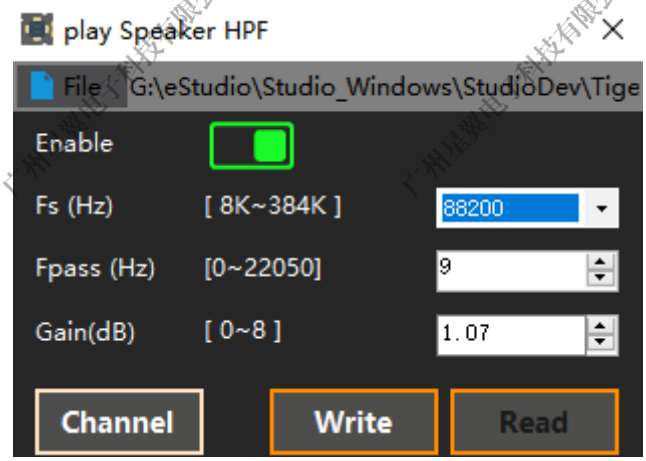

图 6-2: HW-HPF 界面示意图

调参界面由 3 部分组成，分别如下。

1. 音效开关按钮；
2. 音效算法参数设置界面；
3. 与板卡端交互的读写按钮。

音效开关按钮

点击音效开关按钮，当开关背景为 绿色 时，表示使能了对应音效算法功能，否则为屏蔽。

音效算法参数设置界面

根据所需参数进行设置，但每个音效算法所需参数各有差异，具体的音效算法调参界面、参数说明、调参效果见音效算法调参说明 章节。

表 6-1: 图形化调参支持列表

音效算法
EQ
HW-DRC
HW-HPF

与板卡端交互的读写按钮

- “Read” 按钮，从板卡端读取参数 (该功能暂时不支持)；
- “Write” 按钮，用于实时调试时将参数发送给板卡端。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2.2 下发参数

参数调好后，点击界面的调参 “Write” 按钮，会自动将音效参数转化为音效参数配置文件，并将该文件推送至板卡端，最终调用板卡端音效管理架构所提供的实时调参服务应用，完成音效实时调参。

查看新音效参数配置文件

该文件保存在工具相对路径下的/param 文件夹下，文件命名为 audio_{stream}_{sfx}_param.xml。

配置推送至板卡端文件路径

修改 TigerAACT.ini 配置文件中的 [AACT] 节点

```
[AACT]
devicepath=/vendor/etc/sfx_mgmt
parampath=/data/vendor/etc/sfx_mgmt
start_command=sfx_param_adjust -DEV {0} -PARM {1} -FILE {2}
```

- devicepath: 板卡端存放音效设备信息文件的位置；
- parampath: 将音效参数配置文件推动到板卡端的位置；
- start_command: 推送完配置文件之后，启动的命令格式，即实时调参服务应用。

手动执行调参命令

如需要用 adb 手动执行调参命令，则需要在 cmd 命令对话框中输入对应的调参指令，以播放流、硬件 DRC 算法为例，命令如下。

```
adb shell sfx_param_adjust -DEV Speaker -PARM HW-DRC -FILE /data/vendor/etc/sfx_mgmt/audio_play_hw_drc_param.
    xml
```

- {-DEV } 表示播放设备类型或录音设备类型
- {-PARM } 表示音效算法类型
- {-FILE } 表示该算法参数文件路径

6.3 音效调参效果确认

由于不同音效算法，其测试音频文件有所不同，如 EQ 算法需要使用扫频文件进行测试，DRC 算法则需要使用扫幅文件进行测试。正常情况有两种调参效果确认，首先是粗调确认调参是否有效，再微调到所需音效效果。

粗调确认

1. 如 android 系统，打开任意音频播放器播放对应测试音频文件；

2. 调整 2 版音效参数，该 2 版参数需要在听感上能明显差异，如 EQ 算法，第一版参数将所有频段调高，第二版参数将所有频段拉低；
3. 将这 2 版参数逐一下发至板卡，直接从听感上确认实时调参是否生效。

微调确认

打开板卡端音效管理架构的音频文件保存调试功能，并将生成的文件拉到 PC 端，使用音频查看工具确认是否生效 (该功能暂时不支持)。

广州星翼电子科技有限公司zdyzlinux

7 音效算法调参说明

7.1 软件类

远程处理类音效算法包含在软件类算法描述中。

7.1.1 EQ

概述

EQ 均衡器是一种可以分别调节各种频率成分电信号放大量的音效算法，通过对各种不同频率的电信号的调节来补偿扬声器和声场的缺陷，补偿和修饰各种声源及其它特殊作用，严格地说应先要根据音响的频响曲线用均衡器来校正成平直的，就是说音响的频率响应曲线本来不是水平的直线，但是为了真实还原声音，我们可以通过均衡器的调节把原来的曲线调得更为平坦。

本算法的实现参照了 Audition 的参数均衡器，使用了五种滤波器实现频响曲线调节，分别如下。

- **Low_shelving：** shelving 滤波，控制低频段的增益；
- **High_shelving：** shelving 滤波，控制高频段的增益；
- **Low_pass：**低通滤波器；
- **High_pass：**高通滤波器；
- **peaking：** peak 滤波，控制带通内增益。

不同的滤波器由不同的参数进行控制：

1、**Shelving 滤波：**需要截止频率和增益

Frequency(频率) —— 这是用于设定你要进行调整的高频段或者低频段的截至频率点的参数；

Gain(增益) —— 用于调整在你设定好的 F 值上进行增益或衰减的参数。

2、**高/低通滤波：**需要截止频率

Frequency(频率) —— 这是用于设定你要进行高低通滤波的截止频率的参数。

3、**Peak 滤波：**需要频率，增益控制，Q 值

Frequency(频率) —— 这是用于设定你要进行调整的频率点的参数；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Gain(增益) —— 用于调整在你设定好的 F 值上进行增益或衰减的参数；

Quantize(频段宽/频率) —— 用于设定你要进行增益或衰减的频段 “宽度” 的参数，设定的 Q 值越小，所处理的频段就越宽，而当设定的 Q 值越大的时候，所处理的频段就越窄。

调参界面

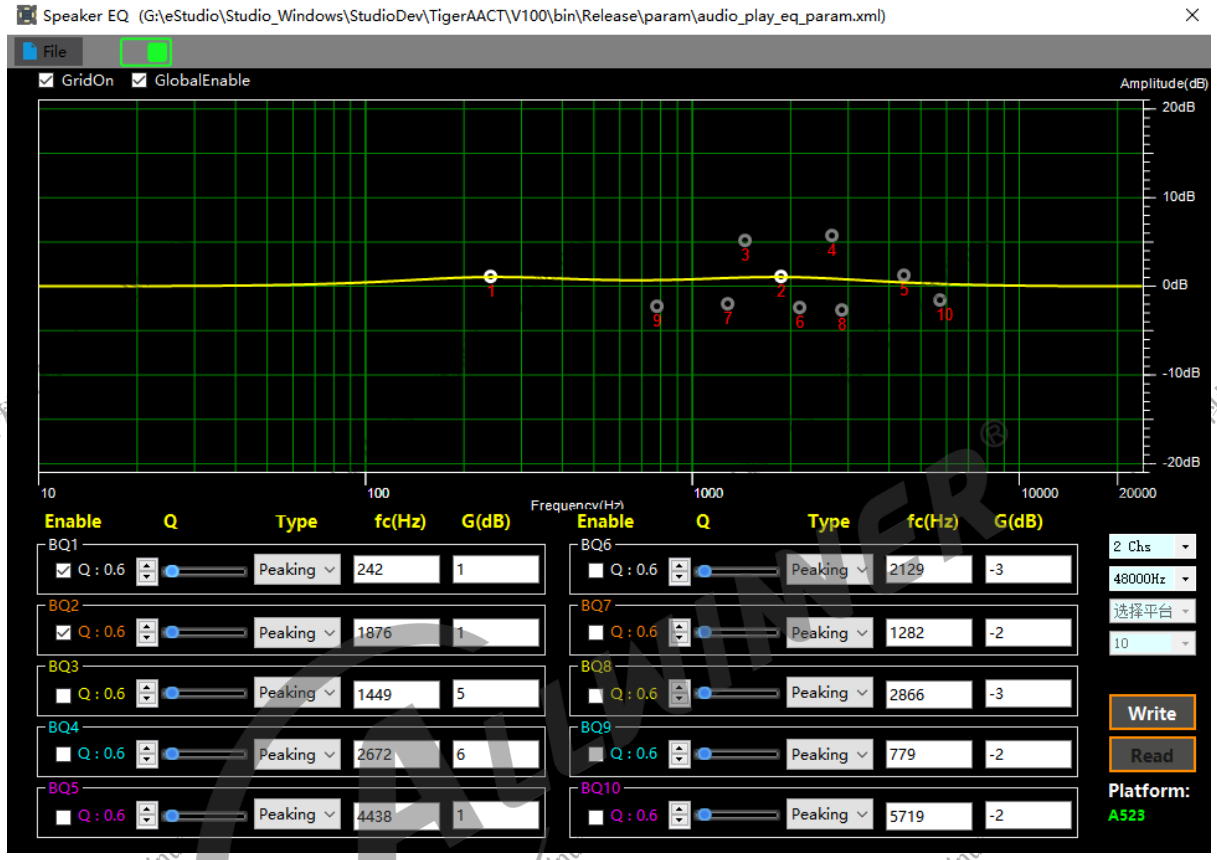

图 7-1: EQ 调参界面

参数说明

可视化界面可支持 10 段的 EQ 参数可视化调节，分上下两部分。

上半图中横坐标为频率，纵坐标为频率的增益。

下半图中依次为配置每一段频点参数，每一段参数如下。

- Enable，使能每一段 EQ；
- Q 量化参数，用于量化每段频点的占空比；
- Type 为滤波器的类型，支持高通、低通、坡峰等；
- fc 为每一段的频点；
- G 为每一段 EQ 的增益。

调参效果

如图所示，为扫频信号，频率从 20-20k 的正弦波，上图为时域波形，下图为频域，表示从低到高的直线。

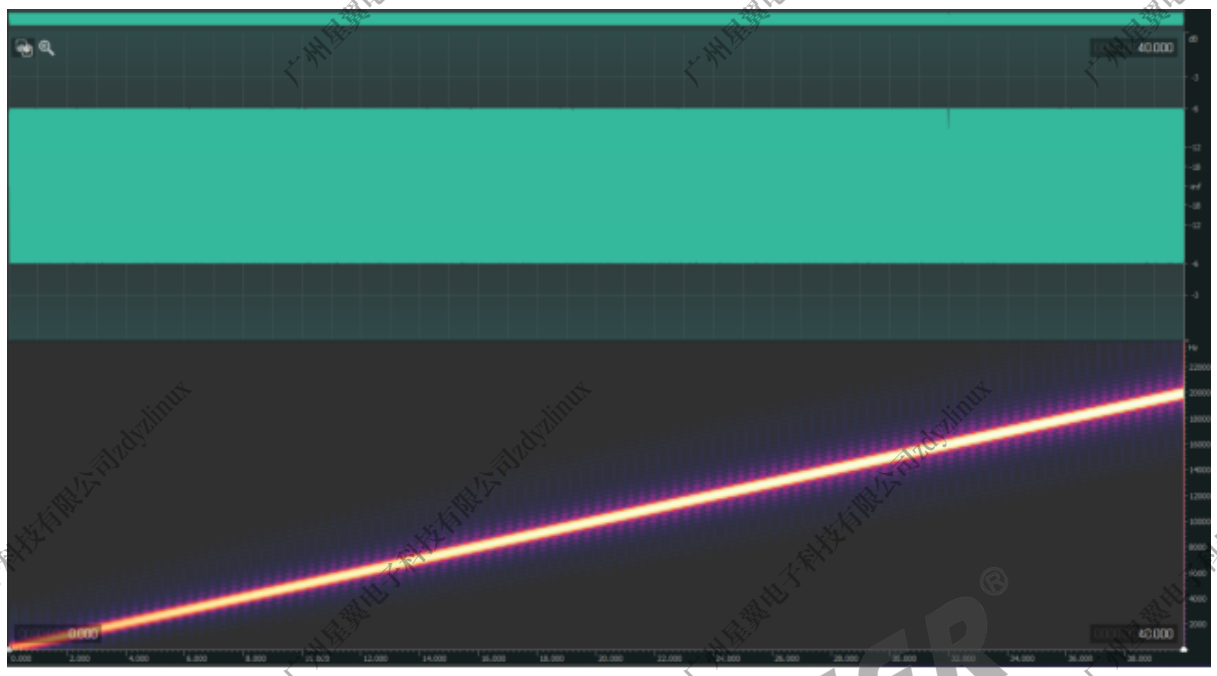

图 7-2: EQ 处理前信号

说明

该界面非 AACT 工具界面，使用常见音频分析软件打开音频文件即可，此处仅说明所使用的音频测试文件内容。

配置 EQ 参数如下。

广州星翼电子科技有限公司zdyzlinux

图 7-3: EQ 的参数配置

EQ 处理后的效果。

广州星翼电子科技有限公司zdyzlinux

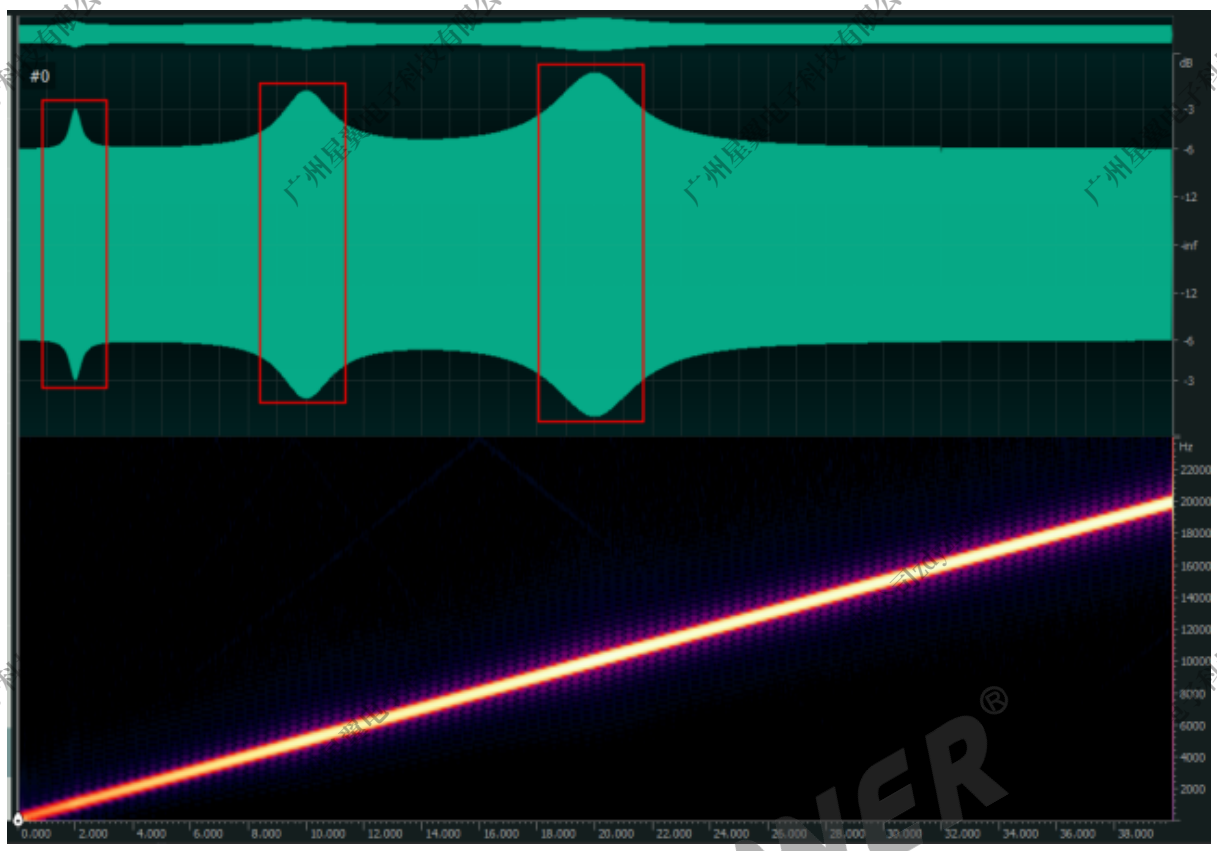

图 7-4: EQ 处理后效果

7.2 硬件类

7.2.1 DRC

概述

DRC，dynamic range control，动态范围控制，常用于音频的输出端，提供音频的压缩和放大能力，可以使小幅值的声音听起来更大声，或者大幅值的语音更加细语柔和，并且部分过滤掉低电平的噪声，是一种非线性的语音信号的幅度调节方式。

比较典型的 DRC 算法，一般将声音的幅值划分为若干子带，侧重于从数字域上对增益进行有选择，有目的的调整。一般小信号底噪会被剪掉 (cutoff)，中间的信号获得合适的一个增强 (amplify)，而幅值比较大的信号将得到抑制处理 (compress)。

DRC 的主要作用如下。

- **保护 DAC 使其不会过载：**当信号发送时，发送器有峰值限制，超过这个峰值会使得发送器过载，DRC 可以压缩原始语音的动态，使得发送信号的动态范围满足发送器的要求，例如音频智能功放中的 DRC 可以实现扬声器的振幅保护；
- **优化信息：**可以对有效人声幅值进行放大处理；

- **压制低电平的噪声：**设置噪声门限，对小信号进行削弱处理，有一定的降噪效果。

调参界面

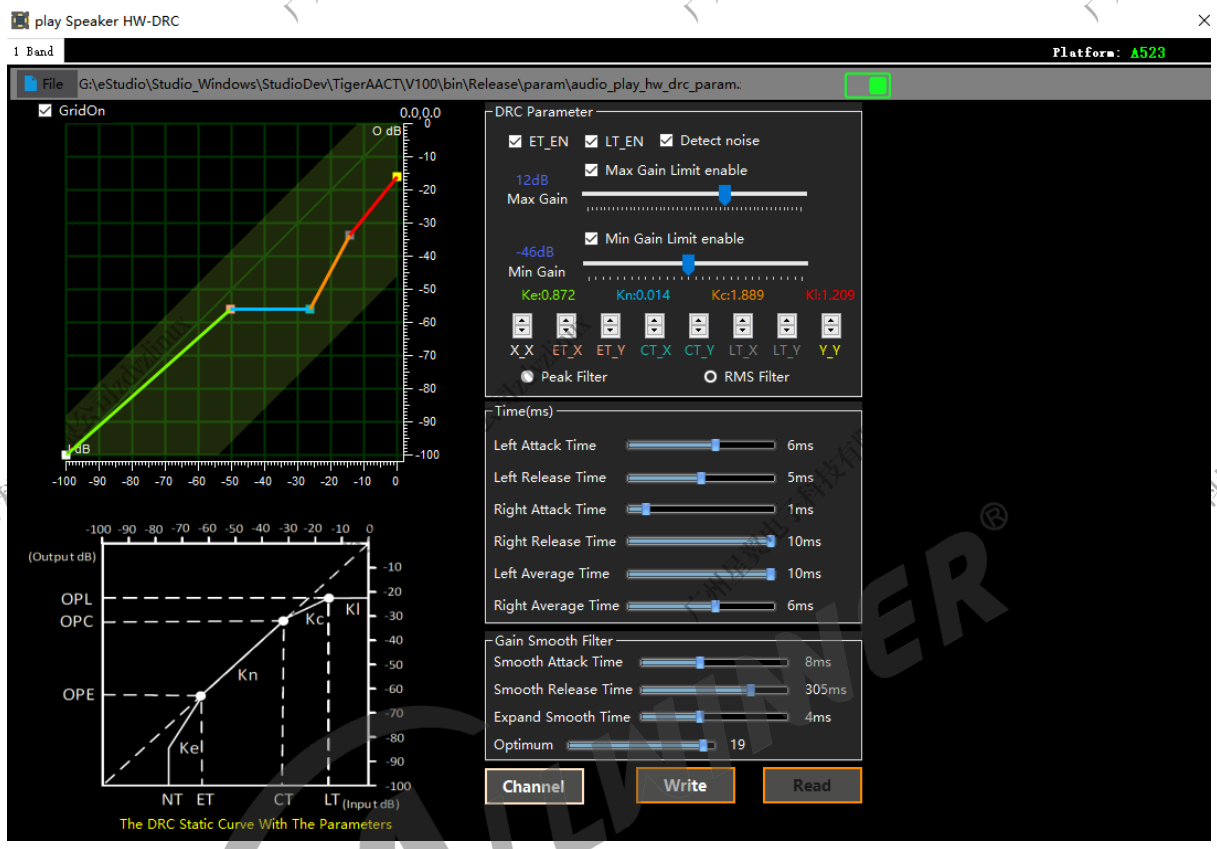

图 7-5: HW-DRC 调参界面

参数说明

参数说明（默认参数如图中所示）：

- **ET_EN：** enhance threshold enable 使能增强门限；
- **LT_EN：** limit threshold enable 使能限制门限；
- **Detect noise：**使能噪声判断；
- **Max_gain：**正增益范围；
- **Min_gain：**负增益范围。

调参效果

DRC 参数曲线如图所示：

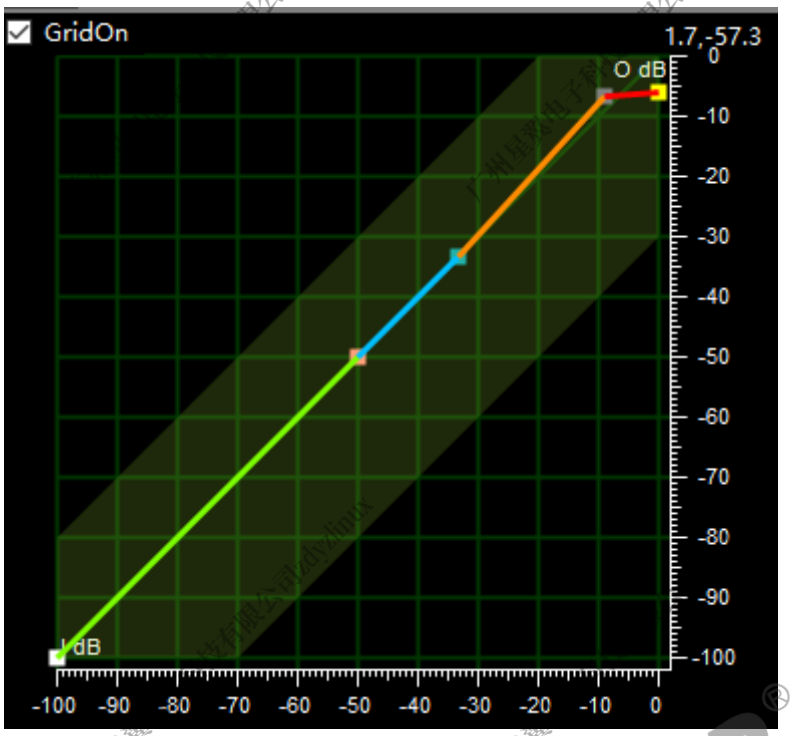

图 7-6: DRC 参数曲线

DRC 输入音频信号：

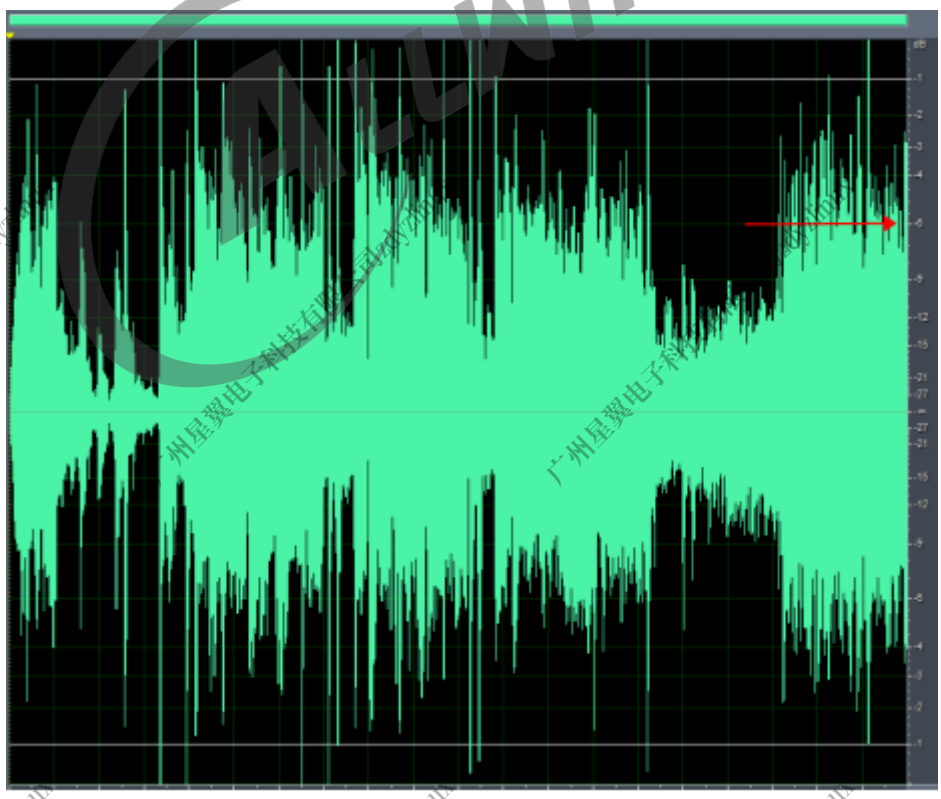

图 7-7: DRC 输入信号

DRC 输出音频信号：

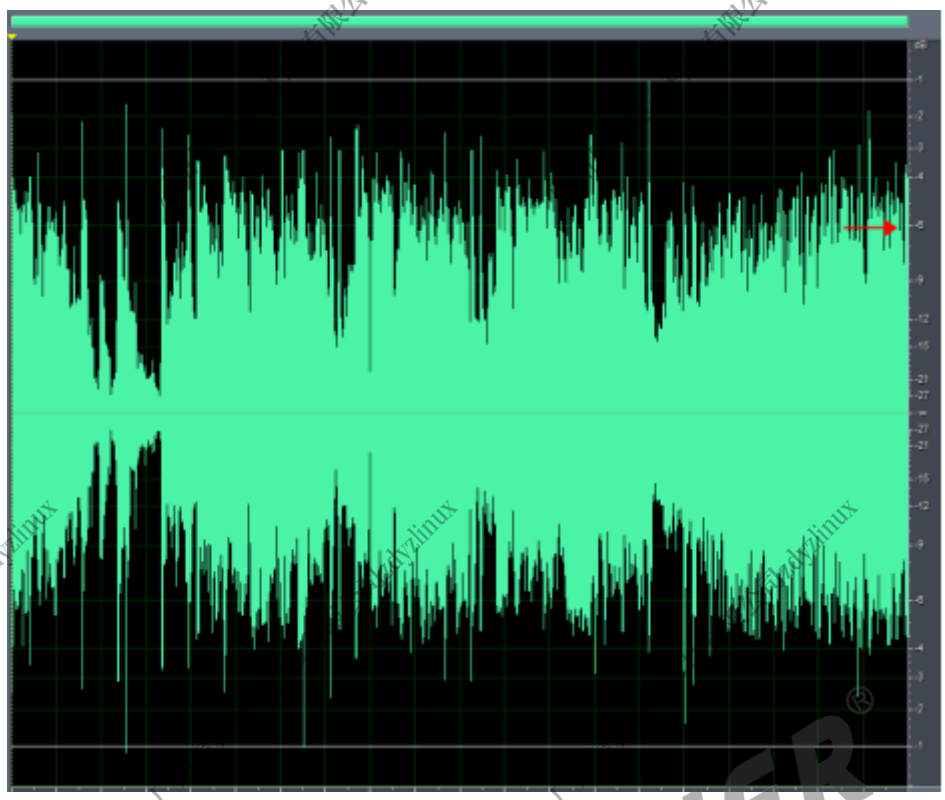

图 7-8: DRC 输出信号

7.2.2 HPF

概述

高通滤波器。

调参界面

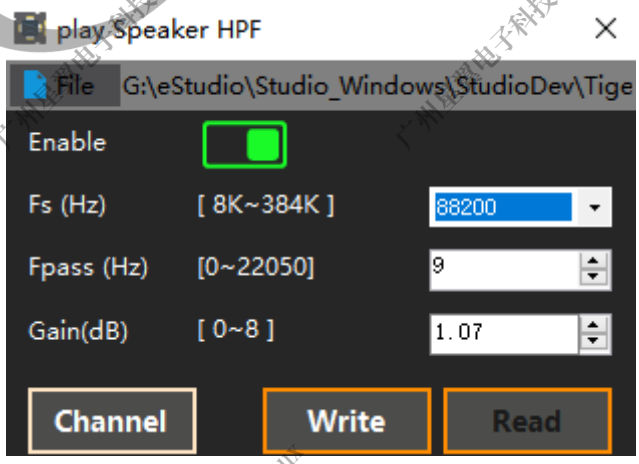

图 7-9: HW-HPF 调参界面

参数说明

- **Enable**：高通滤波器使能；
- **fs**：采样率；
- **fpass**：截止频率；
- **gain**：增益。

调参效果

参数配置示例如下。

fs(32000)，fpass(4000)，gain(1)。

HPF 输入音频信号：

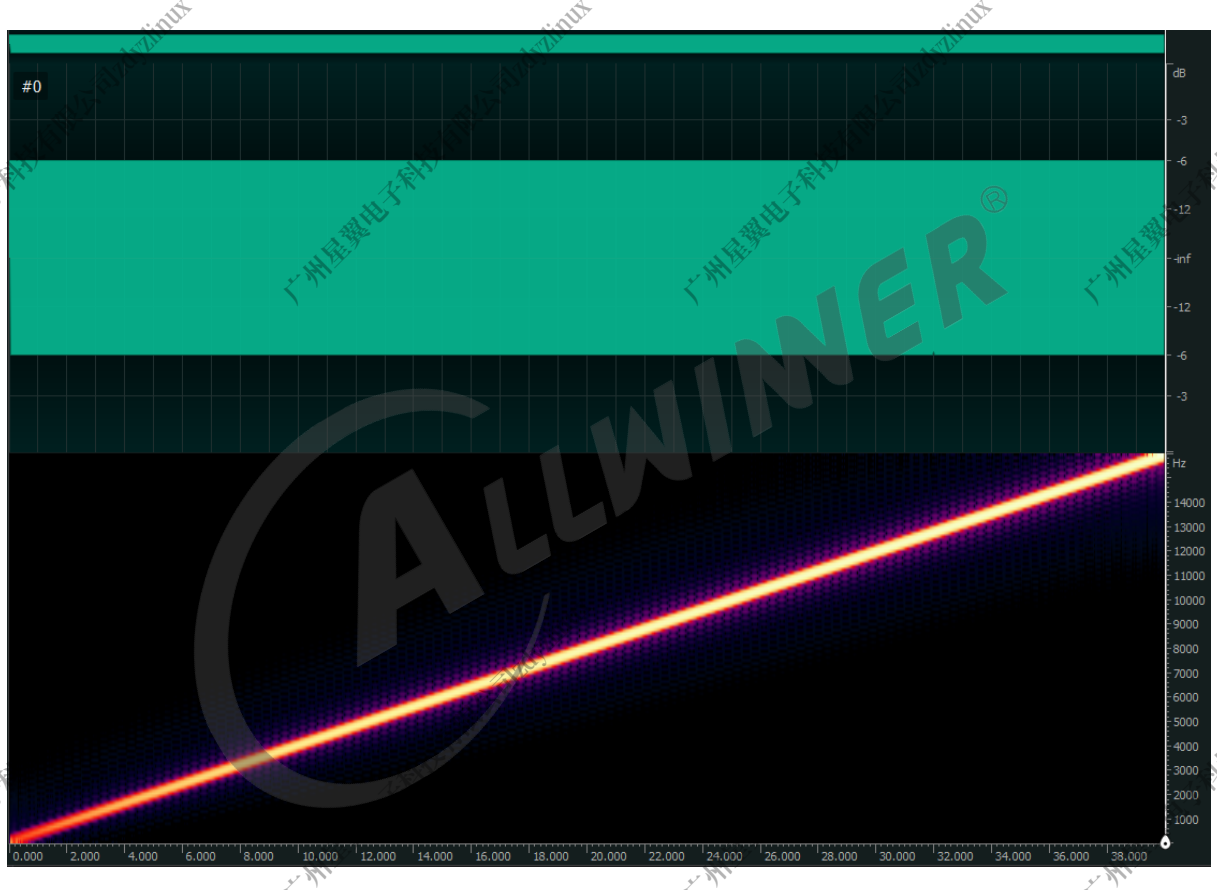

图 7-10: HPF 输入信号

HPF 输出音频信号：

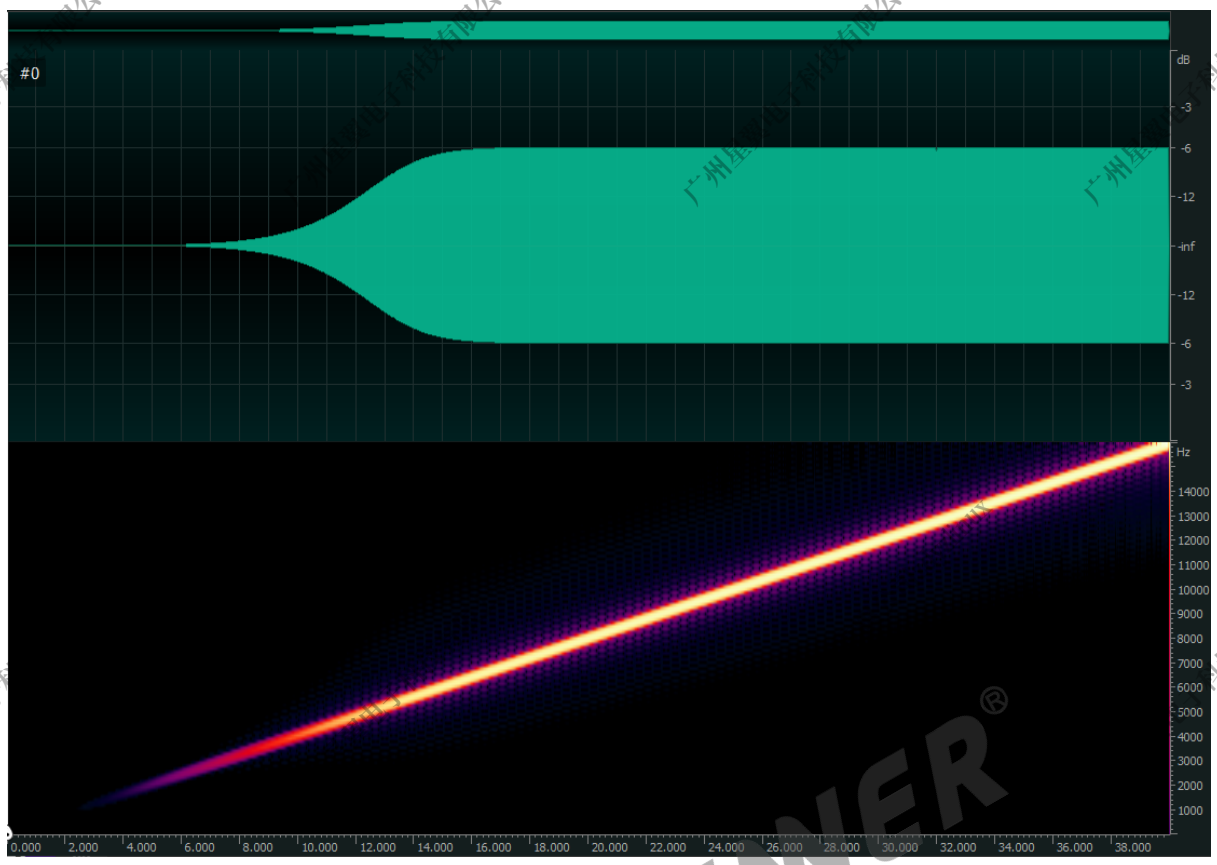

图 7-11: HPF 输出信号

广州星翼电子科技有限公司zdyzlinux

8 FAQ

8.1 Android 系统中上位机设置参数失败

1. 确认板卡所使用 Android 固件已进入 root 模式；
2. 板卡支持 adb，且与上位机正确连接；
3. 板卡需先执行 **“setenforce 0”** 命令，开启文件读取权限。

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。